

Assignment of parcels to loading stations in robotic sorting systems

Xianhao Xu ^a, Yuerong Chen ^a, Bipan Zou ^{b,*}, Yeming Gong ^c

^a School of Management, Huazhong University of Science and Technology, 1037 Luoyu Road, Wuhan, 430074, China

^b School of Business Administration, Zhongnan University of Economics and Law, 182 Nanhu Avenue, Wuhan, 430073, China

^c EMLYON Business School, 23 Avenue Guy de Collongue, 69134 Ecully Cedex, France

ARTICLE INFO

Keywords:

Robotic sorting system
Order assignment
Integer programming
Tabu search
Queueing network

ABSTRACT

Autonomous robots are increasingly used in warehouses in the recent decade, due to their flexible throughput capacity and low operating cost. Sorting may be the newest warehouse scene where autonomous robots are adopted. We consider a robotic sorting system with a two-tier layout where robots drive on the top mezzanine and sort parcels from loading stations (inputs) to drop-off points (outputs) via spiral conveyors connected to roll containers at the lower tier. We investigate the assignment optimization of arrival parcels to loading stations, to minimize the system throughput time. We first build an open queueing network to estimate the system performance and validate its accuracy by simulation. Then, we formulate an integer-programming model that takes the minimization of throughput time as the objective. We prove the computational complexity of the model by transferring it into an order batching problem, and design a Tabu search algorithm for solution. We evaluate the efficiency of the algorithm by both numerical experiments that take the Gurobi solver, the random and closest assignment rules as the comparison target, and a real case study. The results show that our algorithm can reduce the system throughput time by 7.09% and 8.76% and lower the manual cost by 11.99% and 17.50% over the random and the closest assignment rule, respectively. Moreover, it outperforms the Gurobi solver in large instances in terms of throughput time. The real case study shows that the system throughput time and the manual cost can be reduced by about 25% and 16%, compared with the assignment rule used in practice.

1. Introduction

In the recent decade, many robotic systems have been implemented in warehouses to handle inventory replenishment and order picking, packaging and even last-mile delivery. Sorting is a new application of robotic autonomous vehicles in intralogistics, that has seen many implementations, see, e.g., the Amazon unveils Xanthus and Pegasus fulfillment robots (Ames, 2019) and the t-sort solutions of Tompkins (2020). As an example, Deppon Express, one of the largest express delivery companies in Shanghai, China, has implemented such a system on two vertical tiers (Zhang, 2019). The upper tier is a mezzanine with evenly distributed holes (see Fig. 1(a)), the drop-off points, each of which is connected through a spiral conveyor with a roll container installed at the lower tier which collects the items for a destination (see Fig. 1(c)). Parcels are transported by a conveyor-based sorter to the loading stations located at the perimeter of the top tier, where manual workers or automated handlers put the parcels onto the robots. A robot travels in the aisles to the designated drop-off point and releases the parcel.

Compared with an automated conveyor-based sorting system, a robotic sorting system (RSS) enjoys a more flexible layout since the input and output points can be easily changed by modifying the control software, and it provides a more scalable throughput

* Corresponding author.

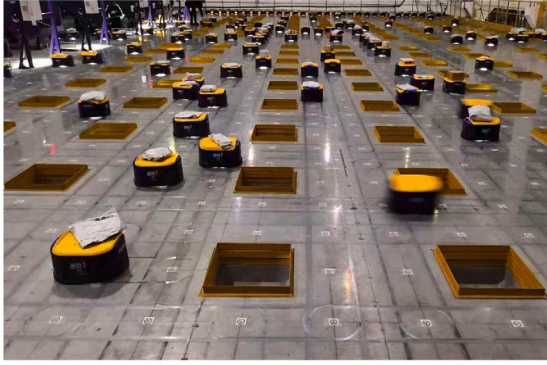
E-mail address: zbp0307@126.com (B. Zou).

<https://doi.org/10.1016/j.tre.2022.102808>

Received 15 October 2021; Received in revised form 7 June 2022; Accepted 13 June 2022

Available online 4 July 2022

1366-5545/© 2022 Elsevier Ltd. All rights reserved.



(a) Top tier



(b) Robot



(c) Bottom tier

Fig. 1. Robotic sorting system (Deppon Express).

capacity by adding or retracting robots. Moreover, organizing the drop-off points on a platform can save much floor space and further reduce the manpower. However, a robotic sorting system generally needs a high investment in both the expensive robots and complex control software. While the design phase before the system implementation generally guarantees a satisfactory throughput performance, the practical performance is largely affected by the operating policies and many fluctuating factors. Based on our research experience on the RSS in the distribution center of Deppon Express in Shanghai, the assignment of arrival parcels to loading stations poses a major challenge for the RSS to produce a good performance. On one side, it determines the waiting time of parcels at loading stations. On the other side, it affects the robot travel time to sort parcels on the platform. However, a simple assignment rule can hardly coordinate them, such as the random assignment rule applied in the RSS of Deppon Express. In this paper, we try to optimize the assignment of parcels to loading stations, depending on the destination information of the parcels. We take the system throughput time (from the parcel entering the system to the parcel releasing into the drop-off points) as the most critical system performance measure, and the maximum service capacity of each loading station as the key constraint (to avoid deadlock at loading stations). We answer the following research questions:

1. How to build an accurate model to capture the system performance of the RSS with a certain assignment rule of parcels to loading stations ?
2. How to assign parcels to the loading stations to minimize the throughput time of a sorting order in the system ?
3. How many loading stations do we need to achieve a required system performance, considering the sorting demand ?

To answer above research questions, we first develop an open queueing network (OQN) that takes the conveyor congestion into consideration, to estimate the system performance of the robotic sorting system. A decomposition method is designed to solve the model and simulation is used for accuracy validation of the OQN. Then, we build an integer-programming model to optimize the assignment of parcels to loading stations, taking the minimization of the order throughput time as the objective function and the maximum service capacity of each loading station as the key constraint. To solve this model, we first use the best fit decreasing (BFD) algorithm to find an initial feasible solution and then implement a Tabu search to improve the solution. The neighbor solutions of the Tabu search are generated by transferring sorting demand assigned to the loading station with large waiting time or delivery time

to other loading stations with sufficient service capacity. We validate the effectiveness of our algorithm by numerical experiments with small, medium and large instances, taking both the order throughput time and the manual cost as the comparison criteria. Both the random and closest assignment rules of parcels to loading stations, and the Gurobi solver are taken as the comparison targets. The results show that, the Tabu search algorithm can lower down the system throughput time by 7.09% and 8.76% and reduce the manual cost by 11.99% and 17.50%, compared with the random and the closest assignment rule, respectively. It produces a better throughput time than the Gurobi solver in large instances, using much less time. We also investigate the effectiveness of our algorithm by the demand data of 30 randomly selected days of the robotic sorting system of Deppon Express, one of the largest express companies in China. The result show that our algorithm can reduce the system throughput time and the manual cost by about 25% and 16% on average, respectively. We at last conduct a sensitivity analysis on the arrival distribution of sorting orders.

This study makes the following contributes: First, we have studied a new operating problem in the robotic sorting system. Second, we have developed an analytical approach that combines a queueing network and an integer programming model to improve the assignment rule of parcels to loading stations in the RSS. A Tabu search algorithm is used for the solution of the model and both numerical experiments and a real case study support the superiority of our approach in both terms of throughput time and manual cost.

The remaining part of the paper is organized as follows: Section 2 presents the literature review. Section 3 gives the modeling preparations, including system description, assumptions and notations, as well as the open queueing model of the RSS. Section 4 builds an integer-programming model for the optimization of the assignment problem of parcels to loading stations in the RSS and designs a heuristic to solve the model. Section 5 validates the accuracy of the open queueing network by simulation. Section 6 includes the numerical experiments and Section 7 concludes the study and gives future work.

2. Literature review

Based on the system and the operating policy examined in this study, we first review the studies on the assignment problems in automatic conveyor-based sorting systems, then concentrate on the studies on assignment problems in robotic storage and order-picking systems. At last, we summarize the results in a table to distinguish the difference between existing research and our study. The order consolidation area or accumulation area in a distribution center generally implements conveyor-based automatic sorting systems, before the introduction of autonomous robots in sorting system. The assignment problems in automatic sorting systems mainly involve the assignment of trucks/orders to induction lines or outbound lanes, and the assignment of orders to the sectors of the conveyor, such as trays. Meller (1997) first examines the assignment of orders to outbound lanes in a two-level order sortation system that combines the warehousing area and the order accumulation area. He builds an integer-programming model that takes the minimization of the completion time of an order wave as the objective function, and design an algorithm by decomposing the problem into a series of minimum-cardinality sub problems. McWilliams (2009) considers a dynamic assignment problem of inbound trailers to the unload docks in a parcel consolidation terminal. He develops an integer-programming model to minimize the workload imbalance cost and designs a dynamic load-balancing algorithm for model solution. Boysen et al. (2010) study the optimization of the assignment problem of inbound trucks to unload docks and that of the outbound trucks to load docks at a cross docking terminal. They have built an integer-programming model and decompose the model into two sub-problems that are solved by a dynamic programming approach and a heuristic, respectively.

Except for the origins or destinations information, many other factors are subsequently investigated in the assignment problems of automatic sorting systems. Acar et al. (2012) take the arrival times and service times of incoming trucks into consideration in the assignment of incoming trucks to unload docks. Van Belle et al. (2013) consider the time window constraints in the assignment of trucks to both inbound and outbound docks. Boysen et al. (2013) consider the penalty for the late arriving shipments in the assignment of trucks at both inbound and outbound docks. Ascó et al. (2014) examine multiple objectives in the assignment problem of available baggage sorting stations to flights which have already been scheduled and allocated to stands, including the maximization of the number of assignment of baggage sorting stations, the minimization of sorting distance and the robustness. Jarrah et al. (2016) also consider multiple objectives in the optimization of the destinations to loading doors assignment in a package sorting center, including the minimization of the number of changes of destination-to-door assignment from one sort to the next, the minimization of the number of loaders at the doors, and the balance of the package volume assigned to each loader.

While previous studies deals largely with the assignment of orders/trucks to inbound/outbound lanes, Briskorn et al. (2017) study the assignment of parcels to trays in a closed-loop tilt tray sortation system. They formulate a short-term deterministic scheduling problem to minimize the makespan of sorting a set of parcels, considering both a fixed assignment and a variable assignment. They find that some simple rules of thumb perform well in most situations, such as the earliest delivery rule. Boysen et al. (2018) consider a warehouse system that links a picking area, an AS/RS and a consolidation area. They optimize the retrieval sequence of bins released from the AS/RS with the target of minimizing the order consolidation time. A mixed integer programming model is built and a dynamic programming-based heuristic is designed for model solution. For a comprehensive review on automatic sortation systems, we refer the readers to Boysen et al. (2019).

We next review the studies on the assignment problems in other robotic storage and retrieval systems, including the robotic mobile fulfillment system (RMFS), the robotic compact storage and retrieval system (RCSRS) and the vertical robotic storage and retrieval system. The assignment problem is also a main research stream of these systems, that mainly involves the assignment of products to storage locations, the assignment of orders/racks to picking stations and the assignment of robots to storage zones.

The assignment of products to storage locations in robotic storage and retrieval systems is either analyzed as the storage policy in a long planning period or optimized in a short period with deterministic demand. As for the first stream, previous studies

Table 1

An summary on the literature on the assignment problems of sortation and robotic warehousing systems.

Reference	System	Robotized	Assignment problem
Meller (1997)	sortation	×	orders to outbound lanes
Ascó et al. (2014), Jarrah et al. (2016)	sortation	×	destinations to loading doors
Briskorn et al. (2017), Boysen et al. (2018)	sortation	×	orders to trays
McWilliams (2009), Boysen et al. (2010), Acar et al. (2012), Van Belle et al. (2013), Boysen et al. (2013)	cross docking	×	trucks to inbound/outbound docks
Lamballais et al. (2017), Zou et al. (2018), Weidinger et al. (2018), Azadeh et al. (2019b)	storage/order-picking	✓	racks/pods to storage locations
Xiang et al. (2018), Li et al. (2020), Lamballais et al. (2020)	storage/order-picking	✓	products to pods
Roy et al. (2019)	storage/order-picking	✓	robots to storage zones
Zou et al. (2017), Boysen et al. (2017), Merschformann et al. (2019), Valle and Beasley (2021), Wang et al. (2021)	storage/order-picking	✓	orders/racks to picking stations
Zou and Chen (2020), Zou et al. (2021)	robotic sortation	✓	orders/robots to loading stations
This study	robotic sortation	✓	parcels to loading stations

generally build stochastic models to estimate the performance of different storage policies and find the appropriate one for the system. Lamballais et al. (2017) build a semi-open queueing network (SOQN) to investigate both the random and zone-based assignment rules in an RMFS. They find that zone-based assignment rule significantly outperforms the random rule in terms of retrieval throughput time. Zou et al. (2018) estimate both throughput and cost performance of both the dedicated and shared assignment of products to storage bins in a robotic compact storage and retrieval system. The results show that while the dedicated assignment rule can provide better throughput performance than the shared assignment rule, the latter can reduce the annual cost by more than 40%. Azadeh et al. (2019b) study the performance estimation and design of a vertical robotic storage and retrieval system, considering the random assignment of products to storage positions. Lamballais et al. (2020) theoretically analyze the assignment of products to pods in an RMFS by optimizing the number of pods per SKU, the ratio of the number of pick stations to replenishment stations, and the replenishment level per pod. Their results support that the order throughput time is the lowest when products are assigned across as many pods as possible. As for the second stream, Weidinger et al. (2018) investigate the optimization problem of assigning racks to storage locations in between the visits of racks to the picking stations in an RMFS. They develop a mixed-integer programming model that takes the minimization of total robot travel distance as the objective function, and design an adaptive large neighborhood search. Their results show that the shortest path assignment policy provides a performance that is very close to the optimal solution. Xiang et al. (2018) optimize the assignment problem of products to pods and the order batching problem in an RMFS in two steps. Different from above studies, Roy et al. (2019) investigate the assignment of robots to storage zones by a closed queueing network. They find that the pooled assignment rule outperforms the dedicated assignment rule in terms of system throughput capacity in a single storage zone, and assigning robots to the least congested zones outperforms the random assignment rule in terms of system throughput capacity. Li et al. (2020) take the robot energy consumption into the objective function of the assignment optimization of products to pods in an RMFS. They first use the temporal association analysis and a clustering approach to identify highly correlated items to be stored in the same rack, then adopt a new turnover-rate based decentralized storage assignment policy in order to avoid AGV blocking.

Several studies investigate the optimization of the assignment problem of orders/racks to picking stations in robotic storage and retrieval systems. Zou et al. (2017) specifically investigate the assignment problem of orders to order-picking stations in an RMFS, considering the random, handling speed-based and a heuristic-based assignment rules. Their results show that the handling speed-based assignment rule significantly outperforms the random assignment rule in terms of retrieval throughput time, and their heuristic can produce a policy close to the optimal one. Boysen et al. (2017) jointly optimize the assignment of both the batches of orders and the racks to the picking station. They build a mixed-integer programming model and designed a decomposition method. Their results support that an optimized order processing can halve the number of robots required, compared to a simple rule-based approach. Merschformann et al. (2019) comprehensively investigate the assignment problems of replenishment orders and picking orders to picking stations, and the assignment problem of pods to storage locations in an RMFS by simulation. Their results show that the pick order assignment rule affects the system throughput the most among all operational policies examined, and the best decision rule provides a unit throughput rate that is twice that provided by the worst one. Valle and Beasley (2021) simultaneously optimize the assignment of orders and pods to picking stations in an RMFS. They have built an integer programming model and developed two heuristics to solve the model. Wang et al. (2022) optimize the assignment problem of mobile racks to picking stations and the scheduling of the racks that are assigned to every picker, to minimize the expected total picking time. They have formulated a stochastic dynamic program model and design an approximate dynamic programming-based branch-and-price solution approach. Their results show that the picking times can be reduced by about 10% by using their approach. We stop here by analyzing the references on robotic storage and order-picking systems, instead, refer the readers to Azadeh et al. (2019a) for a comprehensive review.

To our best knowledge, limited studies on robotic sorting systems can be found. As the pioneer, Zou and Chen (2020) roughly build a semi-open queueing network for performance estimation of the RSS, and then design a simple heuristic to improve the assignment of orders to workstations in terms of system throughput time. However, they do not consider the capacity limitation of each loading station. Zou et al. (2021) investigate the assignment rule of robots to loading stations by a closed queueing network, considering the random, closest, dedicated and shortest queue robot-to-loading-station assignment rules. They find that both the

Table 2
Main notations used in analytical models.

Notation	Description	Notation	Description
W, L	System width and length, by the number of drop-off points	w_a, w_d	Width of an aisle/cross-aisle and a drop-off point (m)
m	Number of drop-off points, $m = W \cdot L$	n	Number of loading stations
a_x, ca_y	aisle and cross-aisle, $x = 0, 1, \dots, W, y = 0, 1, \dots, L$	ls_j	Loading station, $j = 1, 2, \dots, n$
d_i	Drop-off point, $i = 1, 2, \dots, m$	λ_{d_i}	Arrival rate of sorting orders of drop-off point d_i (per hour), $i = 1, 2, \dots, m$
p_{d_i}	Probability of a parcel goes to drop-off point d_i , we have $p_{d_i} = \frac{\lambda_{d_i}}{\sum_{i=1}^m \lambda_{d_i}}, i = 1, 2, \dots, m$	p_{ls_j}	probability of a parcel goes to loading station $ls_j, j = 1, 2, \dots, n$
x_{ls_j}	x -axis coordinate of loading station $ls_j, j = 1, 2, \dots, n$	x_{d_i}, y_{d_i}	Coordinates of drop-off point $d_i, x_{d_i} = i - \lceil \frac{i}{W} \rceil, y_{d_i} = \lceil \frac{i}{W} \rceil, i = 1, 2, \dots, m$
v_m	Robot maximum travel velocity (m/s)	a	acceleration and deceleration rate of robot (m/s ²)
v_c	velocity of the conveyor (m/s)	μ_{ls_j}	Maximum service rate of loading station $ls_j, j = 1, 2, \dots, n$
t_{lu}	Time for a robot to load/unload a parcel (s)	t_i	Time for a robot to take a 90° turn (s)
k_j	whether a worker is arranged to loading station ls_j ($k_j = 1$ for yes and $k_j = 0$ for no), $j = 1, 2, \dots, n$	$z_{i,j}$	whether drop-off point d_i is assigned to loading station ls_j ($z_{i,j} = 1$ for yes and $z_{i,j} = 0$ for no), $i = 1, 2, \dots, m; j = 1, 2, \dots, n$

random and the shortest queue based assignment rules provide a better throughput capacity for a large number of robots. Our study differs from above studies in two aspects: (1) We have build an OQN for performance estimation of the RSS that captures the sorting process on the conveyor. (2) We have taken the service capacity of the loading stations into account in the optimization model of assignment of parcels to loading stations, explicitly analyzed the computational complexity of the optimization model and designed both exact algorithm and heuristic. The OQN provides the system throughput time of the RSS with a given assignment of parcels to loading stations. The integer-programming model and the Tabu search improve the assignment rule in terms of system throughput time. As a summary on the literature review of assignment rule in sortation system and robotic warehousing systems (Table 1), we can distinguish above studies from this paper on three aspects: system function, whether the system is robotized and the assignment problem examined.

3. Performance estimation of robotic sorting systems

In this section, we first present the system description and give the assumptions and notations. Then, we build an open queueing network for performance estimation of the RSSs, and propose a decomposition method to solve the model.

3.1. System description and modeling preparations

The layout of the RSS examined in this paper is depicted in Fig. 2. The top tier is a mezzanine where items are placed on robots at loading stations (inputs), and then the robots transport them to the drop-off points (outputs) in the middle area. The bottom tier holds the roll containers that connect with the drop-off points to collect released items. Without loss of generality, we assume that n loading stations are allocated at the north side of the system, along the sorter system that transports the items from the entrance of the system to the loading stations. The sorter system is equipped with a scanner that determines which loading station a parcel will be sorted to. The middle area of the platform consists of m drop-off points (organized in a $W \times L$ rectangular) and the aisles and cross-aisles for the robot traveling. Each aisle and cross-aisle has two uni-directional lanes with reverse directions, that prevents the robot conflict. The robot will take the shortest path from the loading station to the designated drop-off point, as the blue line of Fig. 2 shows (we will specify the path later).

We first present the main notations used in this paper in Table 2. Then, we specify the steps to fulfill a sorting order designated to drop-off point d_i as follows:

1. An arriving parcel will be transported by the conveyor to loading station $ls_j, j = 1, 2, \dots, n$ based on the assignment rule applied. The worker puts the parcel on the robot and the robot gets the destination information when it passes the scanner.
2. The robot first moves in aisle $a_{x_{ls_j}}$ from loading station ls_j to the intersection between aisle $a_{x_{ls_j}}$ and cross-aisle ca_y (movement m_1), where $y = y_{d_i}$ if $x_{d_i} \leq x_{ls_j}$, otherwise, $y = y_{d_i}$. Then, it takes a 90° turn and moves in the cross-aisle ca_y to the drop-off point d_i (movement m_2).
3. When the robot reaches the drop-off point d_i , it releases the item into the roll container.
4. The robot first moves from the drop-off point d_i to the first upward aisle the robot meets. Then, it takes a 90° turn and moves in the aisle to the parking area (below the sorter system).

We make the following assumptions in this study:

1. The parcels of a specific destination will be assigned to one loading station.

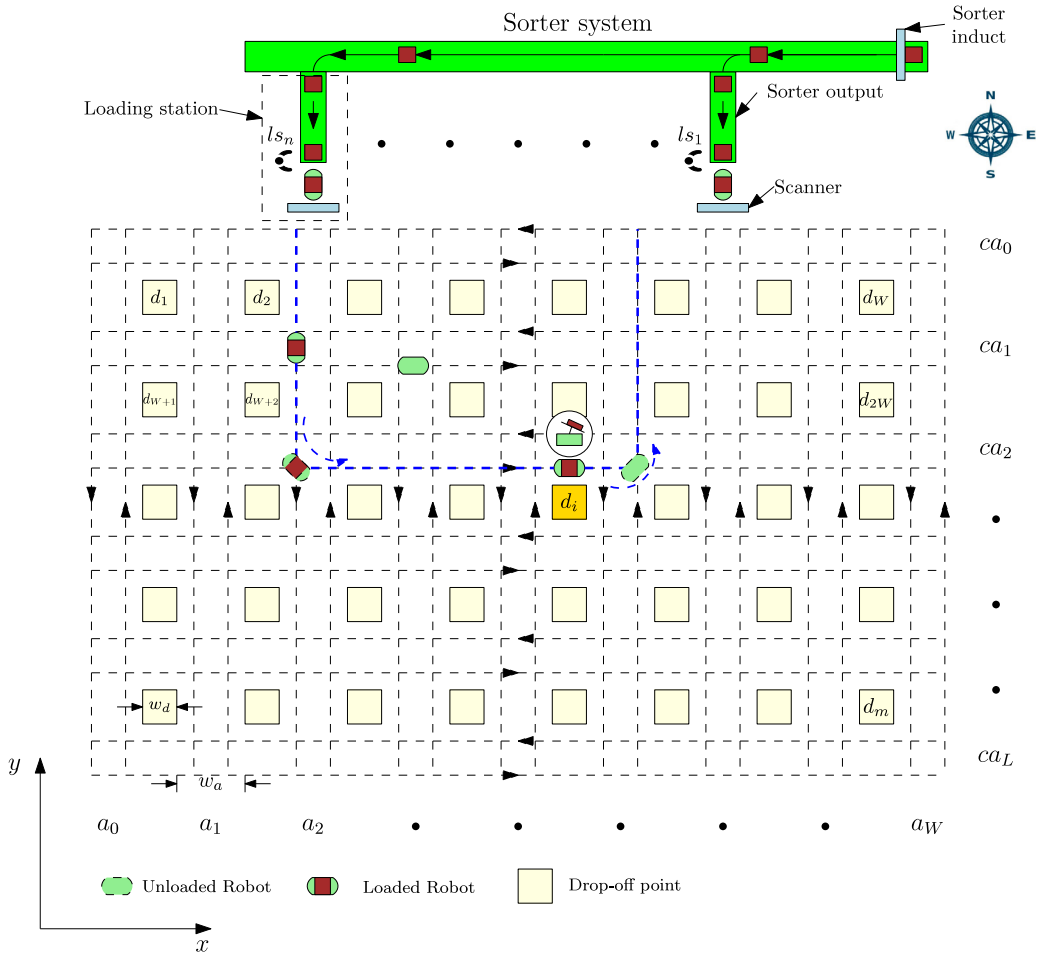


Fig. 2. Layout of the RSS. (For interpretation of the references to color in this figure legend, the reader is referred to the web version of this article.)

- To avoid gridlock at loading stations, the sum of the sorting orders assigned to a loading station cannot exceed the maximum service capacity of the loading station.
- We assume the number of robots is sufficient, such that the waiting time of parcels for robots at the loading stations is ignored. The system will set a large fleet of robots in the design stage, to guarantee a required throughput capacity for the peak demand period, it normally has sufficient robots for the daily operation. Moreover, the loading stations are the bottleneck of the system, the waiting time of the parcel for a robot is generally covered by the waiting time of the parcel for the worker.
- The lanes in aisles and cross-aisles are uni-directional. Under such a setting, we ignore the robot congestion and deadlock in the system. This assumption is made following the existing studies on robotic warehousing systems, including the RMFS (Lamballais et al., 2017, 2020; Roy et al., 2019) and the RSS (Zou et al., 2021). Moreover, existing study on RSS shows that the robot congestion slightly affects the system performance of an RSS with dual-lane aisles and cross-aisles, while the effect becomes significant in the system with single lane aisles and cross-aisles.
- Sorting orders designated for drop-off point d_i arrive at the system following a Poisson distribution with an arrival rate λ_{d_i} (per hour), $i = 1, 2, \dots, m$. We will relax this assumption and conduct a sensitivity analysis on the arrival distribution of sorting orders in numerical experiments.
- The time for a worker at loading station ls_j to put an item on the robot t_{ls_j} follows a uniform distribution $U[t_{ls_j}, \bar{t}_{ls_j}]$. Then, the mean service time and the squared coefficient of variation of the service time is $\mu_{ls_j}^{-1} = \frac{t_{ls_j} + \bar{t}_{ls_j}}{2}$, $scv_{ls_j} = \frac{(\bar{t}_{ls_j} - t_{ls_j})^2}{3(t_{ls_j} + \bar{t}_{ls_j})}$.
- The time for a robot to release an item t_{lu} is constant.
- Idle robots park underneath the sorter system, i.e., a robot waits there for the next order. The parking area of the robot is close to the loading stations, we ignore the robot travel process from the parking area to the loading stations.

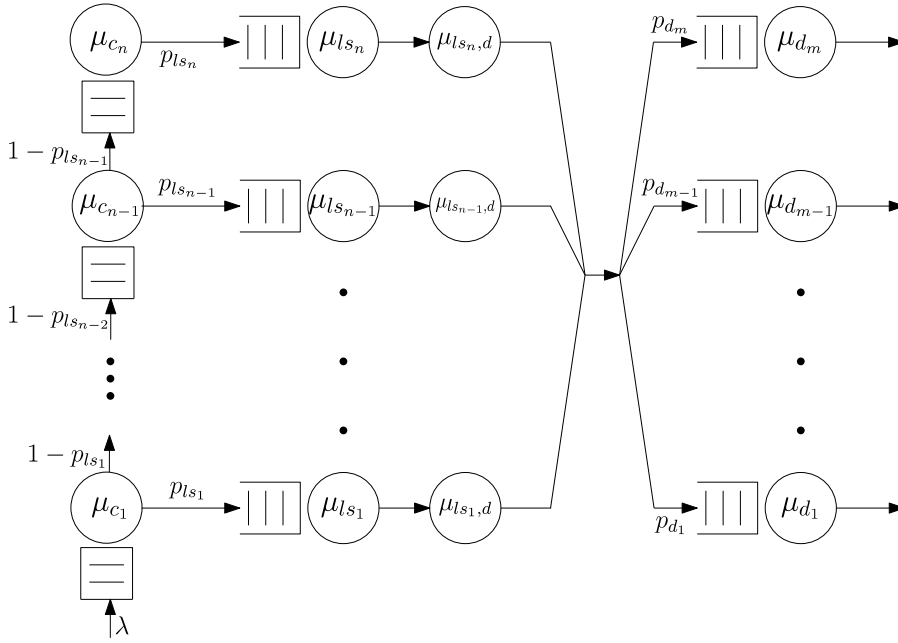


Fig. 3. An open queueing network for RSSs.

3.2. An open queueing network for robotic sorting systems

Based on the fulfillment process of a sorting order and the aforementioned assumptions, we build an open queueing network (OQN) to estimate the performance of RSSs (Fig. 3). The OQN can capture the effect of the assignment of parcels to loading stations on the system performance, including the waiting time of parcels for loading stations and the travel time between the loading stations and the drop-off points. We also build a semi-open queueing network (Appendix B) that can model the waiting process of orders for robots, that is ignored in the OQN.

Parcels first arrive at the conveyor with an integrated arrival rate $\lambda = \sum_{i=1}^m \lambda_{d_i}$. The parcels will be transported by the sorter system to the loading station ls_j with probability p_{ls_j} , $j = 1, 2, \dots, n$, depending on the assignment of parcels to loading stations. The movement of parcels on the sorter system between two neighboring loading stations is modeled as a single service node μ_{c_j} , $j = 1, 2, \dots, n$ with finite queue capacity, due to the possible congestion on the conveyor. The worker puts the parcel on the robot at the service node μ_{ls_j} . Then, the robot transports the parcel from loading station ls_j to the designated drop-off point. The movement process is modeled as an infinite server $\mu_{ls_j,d}$, since no waiting is included. At last, the robot releases the parcel into the drop-off point d_i with probability p_{d_i} , $i = 1, 2, \dots, m$. Each drop-off point is modeled as a single server, since it can be used by one robot once. Note that we ignore the movement of the robot from the drop-off points back to the loading stations in the OQN, since the robots are sufficient (assumption 3). We can also extend the OQN to either a semi-open queueing network or a closed queueing network (depending on the performance measure focused), if the robots are limited resource in the system.

Next, we derive the movement time of parcels on the sorter system $\mu_{c_j}^{-1}$, $j = 1, 2, \dots, n$ and the movement time of parcels on the platform $\mu_{ls_j,d}^{-1}$, $j = 1, 2, \dots, n$ by calculating the travel time from loading station ls_j to drop-off point d_i , i.e., T_{ls_j,d_i} . The movement time of parcels on the sorter system between two neighboring loading stations can be calculated by

$$\mu_{c_j}^{-1} = \begin{cases} \frac{x_{ls_{j+1}} - x_{ls_j}}{v_c}, & \text{if } j < n, \\ \frac{W - x_{ls_n}}{v_c}, & j = n. \end{cases} \quad (1)$$

Once a parcel passes the service node μ_{c_j} , it may go to the loading station ls_j with the probability p_{ls_j} or stay on the conveyor with the probability $1 - p_{ls_j}$.

Based on the fulfill steps of a sorting order, the robot will travel by distance $D_1(ls_j, d_i)$ and $D_2(ls_j, d_i)$ to conduct the movement m_1 and movement m_2 , respectively (Eq. (2)).

$$D_1(ls_j, d_i) = \begin{cases} y_{d_i} \cdot (w_a + w_d), & x_{d_i} \leq x_{ls_j}, \\ (y_{d_i} - 1) \cdot (w_a + w_d), & \text{otherwise}, \end{cases} \quad (2)$$

$$D_2(ls_j, d_i) = |x_{d_i} - x_{ls_j}| \cdot (w_a + w_d)$$

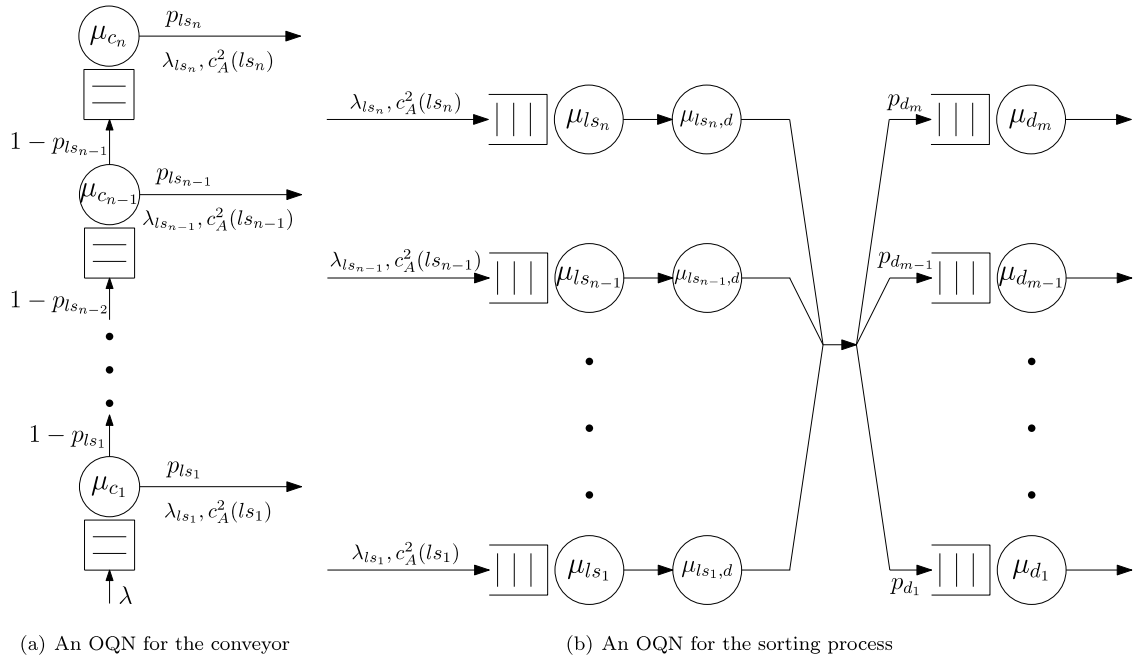


Fig. 4. Split OQNs of original OQN of the RSS.

Take movement m_1 as an example, we can get its travel time by Eq. (3), considering the accelerating and decelerating of the robot. The travel time of the movement m_2 , denoted as $T_2(l_{s_j}, d_i)$, can be obtained by the same way.

$$T_1(l_{s_j}, d_i) = \begin{cases} 2\sqrt{\frac{D_1(l_{s_j}, d_i)}{a}}, & D_1(l_{s_j}, d_i) \leq \frac{v_m^2}{a}, \\ \frac{v_m}{a} + \frac{D_1(l_{s_j}, d_i)}{v_m}, & D_1(l_{s_j}, d_i) > \frac{v_m^2}{a}. \end{cases} \quad (3)$$

Then, the travel time from loading station l_{s_j} to drop-off point d_i can be obtained by $T_{l_{s_j}, d_i} = T_1(l_{s_j}, d_i) + T_2(l_{s_j}, d_i) + t_r$. Considering the probability that a parcel being sorted to drop-off point d_i , i.e., p_{d_i} , we can get the mean value and the squared coefficient variation of the service time at node $\mu_{l_{s_j}, d_i}$ by Eq. (4).

$$\mu_{l_{s_j}, d_i}^{-1} = \sum_{i=1}^m p_{d_i} \cdot T_{l_{s_j}, d_i} + cv_{l_{s_j}, d_i}^2 = \frac{\sum_{i=1}^m p_{d_i} \cdot T_{l_{s_j}, d_i}^2}{\mu_{l_{s_j}, d_i}^{-2}} - 1. \quad (4)$$

In the OQN, the service part of the sorter system (service nodes μ_{c_j} , $j = 1, 2, \dots, n$) has finite queueing capacity and the service nodes $\mu_{l_{s_j}}$ and $\mu_{l_{s_j}, d_i}$, $j = 1, 2, \dots, n$ have non-exponential service times. Since such a queueing network has not product-form solution method (Bolch et al., 2006), we develop a decomposition method that splits the model into two parts for solution. The first part corresponds to the sorter system (Fig. 4(a)) and the second part corresponds to the sorting process on the mezzanine (Fig. 4(b)).

We use the performance estimation procedure of the assembly/disassembly line with the blocking after service blocking rule and general service time (Mountz, 2012) to approximate the performance of the OQN for the sorter system (see the details in Appendix C). We first decompose the OQN into $n-1$ two-station subsystems (Fig. 5), denoted as $(j, j+1)$, $j = 1, 2, \dots, n-1$. Each subsystem has a predecessor station μ_{c_j} , a successor station $\mu_{c_{j+1}}$, and a finite buffer $B_{j,j+1}$ between μ_{c_j} and $\mu_{c_{j+1}}$, $j = 1, 2, \dots, n-1$. The algorithm starts by initializing the system throughput X and some basic parameters. Then, it conducts the following steps until the change in X is within a pre-specified threshold ε : (1). For each subsystem $(j, j+1)$, the algorithm first calculates the first two moments of the virtual arrival rate ($\mu_u(j, j+1)$ and $\zeta_u(j, j+1)$), the first two moments of the departure process ($\mu_d(j, j+1)$ and $\zeta_d(j, j+1)$). (2). Then, it obtains the starving and blocking probabilities at the instance of a service completion at station j ($p_s^*(j, j+1)$, $p_b^*(j, j+1)$) and the starving and blocking probabilities in average-time situation ($p_s(j, j+1)$, $p_b(j, j+1)$). (3). Based on above results, the algorithm can at last updates the system throughput X .

Based on performance measures of the OQN for the sorter system, we can get the expected sojourn time of the parcel on the sorter system, denoted as TH_S , and the order's arrival information to each loading station, including the mean arrival rate $\lambda_{l_{s_j}}$ and the squared coefficient of variation $c_A^2(l_{s_j})$, $j = 1, 2, \dots, n$. Then, we use the decomposition method to analyze the performance measures of the OQN for the sorting process (Bolch et al., 2006; Marchet et al., 2012). This method consists of five sequential steps explained as follows:

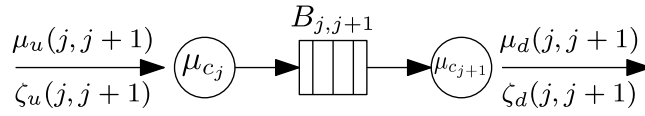


Fig. 5. The queueing model of a two station subsystem of the conveyor.

1. Decompose the original OQN into stochastically independent queues, including $G/G/1(\mu_{ls_j})$, $G/G/\infty(\mu_{ls_j,d})$ and $G/M/1(\mu_{d_i})$, $i = 1, 2, \dots, m, j = 1, 2, \dots, n$.
2. Calculate the arrival rates and utilization of each service node. Specifically, the arrival rate of node $\mu_{ls_j,d}$ is $\lambda_{ls_j,d} = \lambda_{ls_j}$, $j = 1, 2, \dots, n$ and that of node μ_{d_i} is λ_{d_i} , $i = 1, 2, \dots, m$. The utilization of node μ_{ls_j} is $\rho_{ls_j} = \frac{\lambda_{ls_j}}{\mu_{ls_j}}$, $j = 1, 2, \dots, n$ and that of node μ_{d_i} is $\rho_{d_i} = \frac{\lambda_{d_i}}{\mu_{d_i}}$, $i = 1, 2, \dots, m$.
3. Calculate the squared of the variation coefficient of the inter-arrival times and inter-departure times of each service node, denoted as $c_A^2(\cdot)$ and $c_D^2(\cdot)$, respectively. This is done by the following procedures:

- Calculate the squared of the variation coefficient of the inter-departure times of service nodes μ_{ls_j} by Eq. (5) (Whitt, 1983)

$$c_{D,ls_j}^2 = 1 + \rho_{ls_j}^2 (c_{B,ls_j}^2 - 1) + (1 - \rho_{ls_j}^2) (c_{A,ls_j}^2 - 1) = c_{A,ls_j}^2 + \rho_{ls_j}^2 (c_{B,ls_j}^2 - c_{A,ls_j}^2), \quad (5)$$

where c_{B,ls_j}^2 is the squared of the variation coefficient of the service times c_{B,ls_j}^2 .

- Let the squared of the variation coefficient of the inter-arrival times and the inter-departure times of service nodes $\mu_{ls_j,d}$ be $c_{A,ls_j,d}^2 = c_{D,ls_j,d}^2 = c_{D,ls_j}^2$.
- Calculate the squared of the variation coefficient of the inter-arrival times and the inter-departure times of service nodes μ_{d_i} by Eqs. (6) and (7), respectively.

$$c_{A,d_i}^2 = \frac{1}{\lambda_{d_i}} \sum_{j=1}^n c_{D,ls_j,d}^2 \cdot \lambda_{ls_j} \cdot p_{d_i} \quad (6)$$

$$c_{D,d_i}^2 = 1 + \rho_{d_i}^2 (c_{B,d_i}^2 - 1) + (1 - \rho_{d_i}^2) (c_{A,d_i}^2 - 1) = c_{A,d_i}^2 + \rho_{d_i}^2 (c_{B,d_i}^2 - c_{A,d_i}^2), \quad (7)$$

4. Calculate the mean queue length and mean waiting time at each service node. Specifically, the mean queue length and the mean waiting time of nodes μ_{ls_j} can be obtained by

$$\bar{Q}_{ls_j} = \frac{\rho_{ls_j}^2}{1 - \rho_{ls_j}} \cdot \frac{(c_{A,ls_j}^2 + c_{B,ls_j}^2)}{2}, \bar{W}_{ls_j} = \frac{\bar{Q}_{ls_j}}{\lambda_{ls_j}}, j = 1, 2, \dots, n. \quad (8)$$

Those of node μ_{d_i} can be obtained by

$$\bar{Q}_{d_i} = \frac{\rho_{d_i}^2}{1 - \rho_{d_i}} \cdot \frac{(c_{A,d_i}^2 + c_{B,d_i}^2)}{2}, \bar{W}_{d_i} = \frac{\bar{Q}_{d_i}}{\lambda_{d_i}}, i = 1, 2, \dots, m. \quad (9)$$

The throughput time of sorting orders in the robotic sorting systems can be obtained by

$$TH = TH_C + \sum_{j=1}^n p_{ls_j} \cdot (\bar{W}_{ls_j} + \mu_{ls_j}^{-1} + \mu_{ls_j,d}^{-1}) + \sum_{i=1}^m p_{d_i} \cdot (\bar{W}_{d_i} + t_{lu}). \quad (10)$$

4. Optimization model of the assignment of parcels to loading stations in robotic sorting systems

This section first builds an integer programming model to optimize the assignment of parcels to loading stations in robotic sorting systems, taking the system throughput time as the objective function. Then, we analyze the computational complexity of the model and design a Tabu search algorithm for solution.

4.1. An integer programming model

We first take the minimization of the expected throughput time of sorting orders as the optimization objective of the assignment problem of parcels to loading stations in RSSs, and build Model (M.1)

$$\min TH(x_{i,j}, y_j) \quad (M.1)$$

$$\begin{aligned}
s.t. \left\{ \begin{aligned} & \sum_{j=1}^m x_{i,j} < y_j \cdot \mu_{ls_j}, j = 1, 2, \dots, n, & (a) \\ & \sum_{j=1}^n x_{i,j} = 1, i = 1, 2, \dots, m, & (b) \\ & x_{i,j} = 0, 1, i = 1, 2, \dots, m, j = 1, 2, \dots, n, & (c) \\ & y_j = 0, 1, j = 1, 2, \dots, n, & (d) \end{aligned} \right. \quad (11)
\end{aligned}$$

where Constraints (11) ensure the maximum service capacity of loading station ls_j not be violated. Constraints (11) guarantee that the parcels designated for drop-off point d_i will be assigned to one loading station.

We analyze the computational complexity of Model (M.1) by transformation from the order batching problem (OBP), which is well-known to be NP-hard (Gademann and Velde, 2005) and can be stated as follows:

OBP: In an order-picking system with given storage positions, routing strategy and the maximum capacity of the picking device C , how to group a set of customer orders $I = \{1, 2, \dots, i, \dots, m\}$ into a set of batches (candidate batches set is $J = \{1, 2, \dots, j, \dots, n\}$), such that the total distance of the tours required to pick the items is minimized ($\min f = \sum_{j \in J} d_j \cdot y_j$) while the maximum capacity of the picking device is not violated ($\sum_{i \in I} c_j \cdot x_{ij} \leq C$)?

Where x_{ij} is a 0-1 binary that determines whether order i is assigned to batch j , y_j is a 0-1 binary variable that determines whether batch j is chosen, c_j is the capacity required to serve batch j and d_j is the picking tours length of batch j .

In our *RSSOAP*, the sorting orders of drop-off points can be treated as the customer orders in the *OBP*. The maximum capacity of loading stations μ_{ls_j} is equivalent to the maximum capacity of the picking device C . The arrival rate of sorting orders corresponds to the capacity required to serve batch j in the *OBP*, namely, c_j . The constraints on the maximum capacity of loading stations ($\sum_{j=1}^m x_{i,j} < y_j \cdot \mu_{ls_j}, j = 1, 2, \dots, n$) corresponds to the constraint $\sum_{i \in I} c_j \cdot x_{ij} \leq C$ in the *OBP*. As for the objective function of Model (M.1), TH can be regarded as the mean time to serve sorting orders of drop-off point d_i , that works the same way as d_j in the *OBP*.

Therefore, we have Theorem 1

Theorem 1. Both Model (M.1) are NP-hard in the strong sense.

4.2. A heuristic for the optimization model of the assignment problem

In this section, we design a heuristic that combines a best fitting decreasing (BFD) procedure and a Tabu search (TS) algorithm to solve model (M.1). The BFD helps to find a feasible solution and the TS improves the solution.

We first sort the demands of all drop-off points in a descending order, and let the set of the sorted demands be $\lambda_d = \{\lambda'_{d_1}, \lambda'_{d_2}, \dots, \lambda'_{d_m}\}$. Then, we use the following procedure to produce an initial solution S_0 . The procedure keeps assigning the largest demand remaining in the set λ_d into the current loading station. Until the current loading station cannot accept the largest demand remaining, a new loading station is activated. When all sorting demands are assigned, i.e., S_d is empty, the procedure finishes and the initial solution is obtained $S_0 = \{x_{i,j}, y_j, i = 1, 2, \dots, m; j = 1, 2, \dots, n\}$.

Algorithm 1 Best fitting decreasing procedure for the initial solution

- 1: Initialization: Let $k = 1, K = 1, j = 1$.
 - 2: **while** $S_d \neq \emptyset$ **do**
 - 3: Activate the loading station ls_j and let $y_j = 1$.
 - 4: **while** $\sum_{i=k}^K \lambda'_{d_i} < \mu_j$ **do**
 - 5: Assign the sorting demand λ'_{d_i} to the loading station ls_j and let $x_{i,j} = 1$.
 - 6: Remove the sorting demand λ'_{d_i} from the set λ_d .
 - 7: Let $K = K + 1$.
 - 8: **end while**
 - 9: Let $k = K + 1, j = j + 1$.
 - 10: **end while**
 - 11: Output: the initial solution $S_0 = \{x_{i,j}, y_j, i = 1, 2, \dots, m; j = 1, 2, \dots, n\}$.
-

Based on the initial solution obtained by the BFD procedure, we design a Tabu search algorithm to the model (M.1). The algorithm first initializes the Tabu list T , the iteration times be $t = 0$ and gives a value to the maximum iteration times $\bar{t}$. Then, the algorithm sets both the global optimal solution S^* and the current optimal solution as the initial solution S_0 . Next, the algorithm implements the iterative procedure to improve the current optimal solution and the global optimal solution, while the iteration number t is smaller than the maximum iteration times $\bar{t}$. In each iteration, the algorithm first generates a set of neighbor solutions, denoted as $N(S_{current})$, of the current optimal solution $S_{current}$ by two logics specified in Algorithm 3:

- Find the loading station $ls_{j'}$ with the largest waiting time based on the current optimal solution $S_{current}$. Enumerate each sorting demand assigned to loading station $ls_{j'}$, and generate a neighbor solution by transferring the sorting demand to other loading station with sufficient capacity. Add the neighbor solution into the set $N(S_{current})$.
- Find the loading station $ls_{j'}$ with the largest expected delivery time from the loading station to the drop-off points, based on the current optimal solution $S_{current}$. Find the sorting demand with the largest delivery time among all demands assigned to loading station $ls_{j'}$. Generate a neighbor solution by transferring the demand to other loading station with sufficient capacity, and add it into the set $N(S_{current})$.

Based on the set of the neighbor solutions $N(S_{current})$, we find the solutions S_{min} and S_{submin} . If the performance of the solution S_{min} is better than that of the global optimal solution S^* , i.e., $TH(S_{min}) < TH(S^*)$, the algorithm updates the global optimal solution $S^* = S_{min}$ and the current optimal solution $S_{current} = S_{min}$, and adds S^* to the Tabu list T . If the solution S_{min} is not in the Tabu list T , we add the solution S_{submin} to the Tabu list T and update the current optimal solution by $S_{current} = S_{submin}$. Otherwise, we add the solution S_{min} to the Tabu list T , and update the current optimal solution by $S_{current} = S_{min}$.

Algorithm 2 Tabu search

- 1: Initialization: The Tabu list T , the iteration times $t = 0$ and the maximum iteration times $\bar{t}$.
 - 2: Set both the global optimal solution S^* and the current optimal solution $S_{current}$ as S_0 and add S_0 to T .
 - 3: **while** $t < \bar{t}$ **do**
 - 4: Generate the neighborhood $N(S_{current})$ of the current optimal solution $S_{current}$ by Algorithm 3.
 - 5: Calculate the objective value of each solution in the set $N(S_{current})$ by Eq. (M.1). Find the solution S_{min} with the minimum objective value and the solution S_{submin} with the second minimum objective value.
 - 6: **if** $TH(S_{min}) < TH(S^*)$ **then**
 - 7: Let $S^* = S_{min}$, $S_{current} = S_{min}$ and add S^* to the Tabu list T .
 - 8: **else if** $S_{min} \in T$ **then**
 - 9: Add the suboptimal solution S_{submin} to the Tabu list T , and let $S_{current} = S_{submin}$.
 - 10: **else**
 - 11: Add S_{min} to the Tabu list T , and let $S_{current} = S_{min}$.
 - 12: **end if**
 - 13: Let $t = t + 1$.
 - 14: **end while**
 - 15: Output the global optimal solution S^* .
-

Algorithm 3 Procedure used to produce the neighborhood $N(S_{current})$ of current solution

- 1: Find the loading station $ls_{j'}$ with the largest expected waiting time in the current solution $S_{current}$.
 - 2: Calculate the remaining service capacity of each loading station except for $ls_{j'}$, denoted as $\mu'_j, j \in J/j'$.
 - 3: **for** each sorting demand λ assigned to the loading station $ls_{j'}$ **do**
 - 4: **for** $j \in J/j'$ **do**
 - 5: **if** $\lambda < \mu'_j$ **then**
 - 6: Generate a new neighbor solution by transferring the sorting demand λ to loading station ls_j .
 - 7: Add above neighbor solution into $N(S_{current})$.
 - 8: **end if**
 - 9: **end for**
 - 10: **end for**
 - 11: Find the loading station $ls_{j'}$ with the largest expected delivery time $\bar{T}_{ls_{j'}, d_{i,j}}$ based on the current optimal solution $S_{current}$.
 - 12: Find the sorting demand λ with the largest delivery time among all sorting demands assigned to the loading station $ls_{j'}$.
 - 13: Calculate the remaining service capacity of each loading station except for loading station $ls_{j'}$, denoted as $\mu'_j, j \in J/j'$.
 - 14: **for** $j \in J/j'$ **do**
 - 15: **if** $\lambda < \mu'_j$ **then**
 - 16: Generate a new neighbor solution by transferring the sorting demand λ to the loading station ls_j .
 - 17: Add above neighbor into the set of the neighborhood $N(S_{current})$.
 - 18: **end if**
 - 19: **end for**
-

5. Simulation validation of the open queueing network

In this section, we examine the accuracy of the open queueing network by simulation. We build the simulation model by Arena (version 14.7) and put the details in [Appendix A](#). While the analytical model can estimate the throughput time of the RSS with a given assignment rule in a short period, it generally produces some estimation errors due to the assumptions made. We need to check the accuracy of the analytical model before using it in the optimization model. Simulation is a popular way to do this ([Azadeh](#)

Table 3
Parameters of the robotic sorting system for effectiveness validation.

w_a (m)	w_d (m)	v_m (m/s)	a (m/s ²)	v_c (m/s)	t_{lu} (s)	t_l (s)	t_{lsj} (s)
0.6	0.6	2	1	2	2	1	U [5, 10]

Note: The parameters are obtained from the robotic sorting system of Deppon Express, an express company in Shanghai, China.

Table 4
System scenarios for simulation validation.

Scenario	W	L	n	λ_d (per hour)
1	8	8	4	U [5, 65]
2	8	8	4	U [8, 68]
3	8	8	4	U [10, 70]
4	12	10	8	U [8, 65]
5	12	10	8	U [10, 70]
6	12	10	8	U [13, 73]
7	16	12	12	U [5, 65]
8	16	12	12	U [6, 68]
9	16	12	12	U [10, 70]

Table 5
Simulation validation results for the system using the random assignment of parcels to loading stations.

Scenario	Throughput time			Waiting time at loading stations			Utilization of loading stations		
	TH_A (s)	TH_S (s)	δ_{TH} (%)	$\overline{W}_{ls}^A$ (s)	$\overline{W}_{ls}^S$ (s)	δ_W (%)	ρ_{ls}^A (%)	ρ_{ls}^S (%)	δ_p (%)
1	32.69	32.24	1.40	9.22	9.24	0.22	77.78	77.80	0.03
2	38.28	37.41	2.33	14.30	14.39	0.63	84.44	84.49	0.06
3	44.94	42.33	6.17	21.07	21.12	0.24	88.89	88.89	0.00
4	35.27	34.13	3.34	9.40	9.50	1.05	78.13	78.25	0.15
5	40.05	38.71	3.46	13.17	13.23	0.45	83.33	83.39	0.07
6	48.53	47.23	2.75	22.65	22.50	0.67	89.58	89.52	0.07
7	45.08	43.25	4.23	13.17	13.30	0.98	83.33	83.42	0.11
8	45.63	43.44	5.04	14.30	14.23	0.49	84.44	84.40	0.05
9	51.53	49.28	4.57	21.07	20.59	2.33	88.90	88.80	0.11

et al., 2019a; Lamballais et al., 2017; Roy et al., 2016). However, we can hardly use simulation to get the system throughput time in the optimization model, since it is extremely time consuming to check many assignment scenarios of parcels to loading stations.

We consider nine system scenarios (Table 4) for the simulation validation with the system parameters presented in Table 3. We adopt both the random and closest assignment rules of parcels to loading stations. Note the assignment of parcels to loading stations following the closest assignment rule may violate the constraint on the maximum service capacity of a loading station. To deal with this issue, we will assign the demand to its secondary closest loading station if the closest one reaches its capacity. For each system scenario, the simulation model runs 30 replications with a warm-up period of 24 h and a running time of 200 h per replication, leading to 95% confidence intervals where the half-width is within 2% of the average. We collect three performance measures, including the throughput time $TH(s)$, the utilization of loading stations $\rho_{ls}(\%)$ and the expected waiting time at loading stations $\overline{W}_{ls}(s)$. The accuracy of the analytical models is measured by the absolute relative errors of analytical results to simulation results, denoted by $\delta_{TH}(\%)$.

$$\delta_{TH} = \frac{|TH_A - TH_S|}{TH_S} \times 100\%, \quad (12)$$

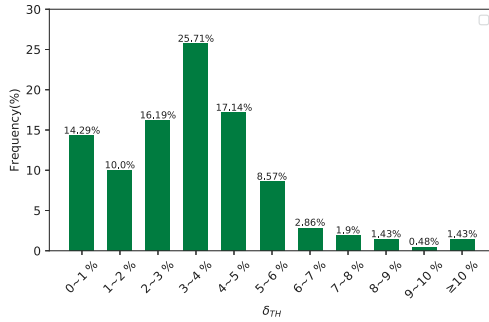
where TH_A and TH_S are the analytical throughput time and the average simulation throughput time, respectively.

We present the validation results of the system using the random and the closest assignment rules of parcels to loading stations in Table 5 and Table 6, respectively. Moreover, we depict the distributions of relative errors under the random and closest assignment rule in Fig. 6 and Fig. 7, respectively. We have the following observations based on the results:

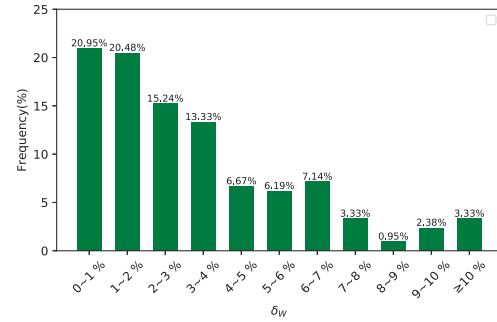
1. The relative errors (average value) of both the throughput time and the waiting time are smaller than 8%, and that of the utilization of loading stations is smaller than 1%. Note that we use the average utilization of all loading stations in both Tables 5 and 6 for each system scenario, while the distributions of the relative errors of utilization use the detailed utilization of each loading station for each system scenario (Figs. 6 and 7). The results support that our analytical model can estimate the system performance of RSS with accuracy.
2. The closest assignment rule outperforms the random assignment rule in both terms of throughput time and waiting time. This implies that we can improve the system performance of an RSS by improving the assignment of parcels to loading stations.

Table 6
Simulation validation results for the system using the closest loading station assignment rule.

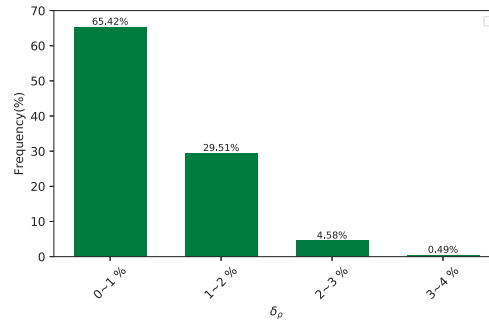
Scenario	Throughput time			Waiting time at loading stations			Utilization of loading stations		
	TH_A (s)	TH_S (s)	δ_{TH} (%)	$\bar{W}_{Is}^A$ (s)	$\bar{W}_{Is}^S$ (s)	δ_W (%)	ρ_{Is}^A (%)	ρ_{Is}^S (%)	δ_ρ (%)
1	29.53	30.9	4.43	9.32	9.36	0.43	77.78	77.78	0.00
2	36.62	38.11	3.91	15.34	15.57	1.48	84.44	84.48	0.05
3	42.71	43.78	2.44	22.27	22.09	0.81	88.89	88.88	0.01
4	30.58	31.85	3.99	9.54	9.58	0.42	78.13	78.18	0.06
5	34.08	35.26	3.35	13.53	13.53	0.00	83.33	83.35	0.02
6	47.58	48.81	2.52	25.86	25.87	0.04	89.58	89.5	0.09
7	41.92	44.51	5.82	17.92	19.34	7.34	83.33	83.39	0.07
8	45.08	47.38	4.85	20.1	21.3	5.63	84.44	84.51	0.08
9	49.23	50.99	3.45	25.68	26.26	2.21	88.9	88.95	0.06



(a) Throughput time



(b) Waiting time at loading stations



(c) Utilization of loading stations

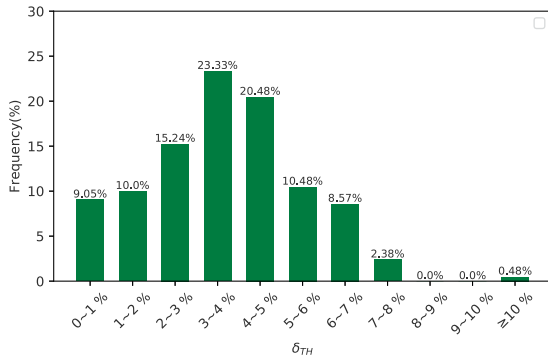
Fig. 6. Distribution of relative errors of analytical results to simulation results under the random assignment rule.

6. Numerical experiments

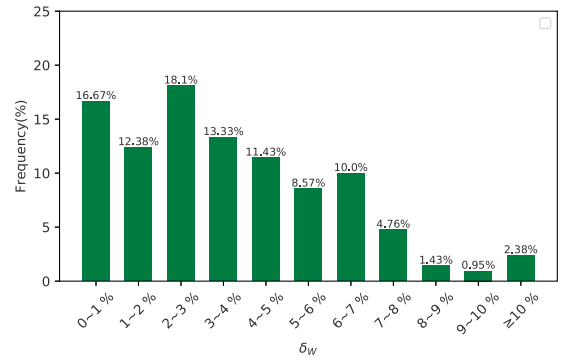
In this section, we first evaluate the effectiveness of the Tabu search algorithm by both a series of numerical experiments and a real case, then we examine the effect of the arrival distribution of sorting orders on the system performance.

6.1. Evaluating TS by numerical experiments

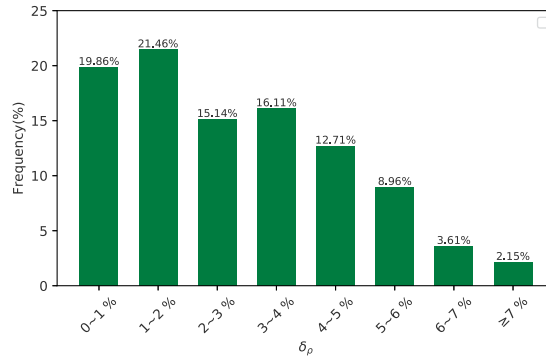
In this section, we evaluate the effectiveness of the Tabu search algorithm by a full factorial experiment, with the basic parameters of the robotic sorting systems presented in Table 3 and system scenarios presented in Table 7. We first divide the factors into two parts: the system design factor (including the system size and the number of loading stations) and the demand factor (the arrival rate of sorting orders). Then, we adopt three levels for each factor, corresponding to low, medium and high level. Specifically, we generate 12 system designs by varying the system size W , L and the maximum number of loading stations n available in the system. The system designs 1,2,3 and 4 refer to small instances, the system designs 5,6,7 and 8 refer to medium instances and the remaining system designs refer to large instances. For each system design, we adopt three levels of arrival rate of sorting orders (indexed by 1,2,3), that makes the utilization of loading stations locate in 70% ~ 80%, 80% ~ 90% and 90% ~ 95%, respectively. Thereby, we



(a) Throughput time



(b) Waiting time at loading stations



(c) Utilization of loading stations

Fig. 7. Distribution of relative errors of analytical results to simulation results under the closest assignment rule.

have 36 instances in total, each of which is indexed as $i - j$, where $i = 1, 2, \dots, 12$ indexes the system design and $j = 1, 2, 3$ indexes the demand level. Moreover, to alleviate the effect of the arrival rate of sorting orders on the system performance, we generate 100 sets of arrival rate for each instance. Note that the system may use workers less than n , depending on the assignment result of destinations to loading stations. Under the random assignment rule, the system assigns a worker to each of the n loading stations, and assigns the destinations to these workers. Under the closest assignment and the TS-based assignment rule, there may be less workers than n .

For each small instance, we first get the optimal assignment rule of parcels to loading stations by enumerating all feasible solutions. Then, we compare its throughput time and the calculation time with those of the Tabu search algorithm, with the result presented in Table 8. The improvement percentage of the throughput time of the optimal assignment rule over that of the Tabu search-based assignment rule is calculated by Eq. (13)

$$\epsilon_T^O = \frac{TH_T - TH_O}{TH_O} \cdot 100\%, \quad (13)$$

where TH_T, TH_O is the throughput time of the Tabu search-based assignment rule and that of the optimal assignment rule, respectively.

We calculate the confidence intervals of the system throughput time over the 100 demand sets with a 95% confidence level. It shows that we can obtain a stable throughput time result. The Tabu search algorithm produces an assignment rule with a throughput time close to that of the optimal assignment rule.

Due to the large calculation time required to get the optimal assignment rule, we next take different assignment rules as the comparison target of the Tabu search algorithm in medium and large instances. We first compare the throughput time of the Tabu search algorithm with that of the random and closest assignment rule, with the results presented in Table 9 and Table 10, respectively. Then, we compare the Tabu search algorithm with the Gurobi solver with a truncated running time 3600 s in terms of system throughput time, with the results presented in Table 11. Moreover, we also compare the manual cost of the Tabu search-based assignment rule with that of the random and closest assignment rule, with the results presented in Table 12.

We have the following observations based on the comparison results in the medium and large instances:

Table 7

System scenarios examined for the evaluation of TS.

Small instances												
Inst.	1-1	1-2	1-3	2-1	2-2	2-3	3-1	3-2	3-3	4-1	4-2	4-3
W	2	2	2	3	3	3	2	2	2	2	2	2
L	4	4	4	4	4	4	6	6	6	8	8	8
n	2	2	2	2	2	2	3	3	3	4	4	4
λ_d (/h)	U [6, 65]	U [8, 68]	U [10, 70]	U [5, 65]	U [8, 68]	U [10, 70]	U [6, 65]	U [8, 68]	U [10, 70]	U [5, 65]	U [8, 68]	U [10, 70]
Medium instances												
Inst.	5-1	5-2	5-3	6-1	6-2	6-3	7-1	7-2	7-3	8-1	8-2	8-3
W	8	8	8	10	10	10	12	12	12	14	14	14
L	5	5	5	6	6	6	7	7	7	8	8	8
n	3	3	3	5	5	5	6	6	6	9	9	9
λ_d (/h)	U [5, 65]	U [8, 68]	U [10, 70]	U [5, 65]	U [8, 68]	U [10, 70]	U [5, 65]	U [8, 68]	U [10, 70]	U [5, 65]	U [8, 68]	U [10, 70]
Large instances												
Inst.	9-1	9-2	9-3	10-1	10-2	10-3	11-1	11-2	11-3	12-1	12-2	12-3
W	16	16	16	18	18	18	20	20	20	22	22	22
L	9	9	9	10	10	10	11	11	11	12	12	12
n	12	12	12	15	15	15	16	16	16	18	18	18
λ_d (/h)	U [5, 65]	U [8, 68]	U [10, 70]	U [5, 65]	U [8, 68]	U [10, 70]	U [5, 65]	U [8, 68]	U [10, 70]	U [5, 65]	U [8, 68]	U [10, 70]

Table 8

Throughput time comparison between the Tabu search assignment rule and the optimal assignment rule in small instances.

Inst.	Optimal (s)			TS (s)			Improvement (%)		
	TH_O	CI_{TH_O}	CPU	TH_T	CI_{TH_T}	CPU	ϵ_T^O	$\bar{\epsilon}_T^O$	$\bar{\epsilon}_T^O$
1-1	7.01	[6.84, 7.18]	171.36	7.01	[6.92, 7.10]	0.23	0.00	0.00	0.00
1-2	7.14	[7.02, 7.26]	171.99	7.15	[7.07, 7.23]	0.25	0.14	0.12	0.16
1-3	7.26	[7.18, 7.34]	172.64	7.27	[7.22, 7.32]	0.26	0.14	0.07	0.21
2-1	7.59	[7.41, 7.77]	2075.19	7.61	[7.55, 7.67]	0.36	0.26	0.24	0.29
2-2	11.35	[11.25, 11.45]	2078.41	11.39	[11.32, 11.46]	0.38	0.35	0.31	0.39
2-3	11.45	[11.30, 11.60]	2079.02	11.49	[11.41, 11.57]	0.39	0.35	0.31	0.39
3-1	7.18	[7.02, 7.34]	2077.39	7.21	[7.12, 7.30]	0.35	0.42	0.35	0.49
3-2	7.25	[7.10, 7.40]	2077.39	7.29	[7.23, 7.35]	0.35	0.55	0.45	0.65
3-3	7.33	[7.13, 7.53]	2078.06	7.36	[7.26, 7.46]	0.38	0.41	0.33	0.49
4-1	9.27	[9.18, 9.36]	17788.34	9.31	[9.24, 9.38]	0.57	0.43	0.41	0.49
4-2	9.29	[9.10, 9.48]	17789.02	9.32	[9.24, 9.40]	0.59	0.32	0.28	0.37
4-3	9.34	[9.23, 9.45]	17778.33	9.39	[9.33, 9.45]	0.58	0.54	0.52	0.55

Note: TH is the mean throughput time over 100 demand sets, CI_{TH} is the confidence interval of the throughput time with a 95% confidence level. CPU refers to the calculation time and ϵ_T^O refers to the minimum and maximum improvement percentage, respectively.

Table 9

Comparison between the Tabu search-based assignment rule and the random and closest assignment rules in medium instances.

Inst.	Random (s)			Closest (s)			TS (s)			Improvement percentage (%)			
	TH_R	CI_{TH_R}	CPU	TH_C	CI_{TH_C}	CPU	TH_T	CI_{TH_T}	CPU	ϵ_T^R	$CI_{\epsilon_T^R}$	ϵ_T^C	$CI_{\epsilon_T^C}$
5-1	14.19	[12.96, 15.42]	0.01	13.45	[12.00, 14.90]	0.03	13.14	[3.21, 5.87]	3.77	7.40	[5.97, 8.83]	2.30	[0.99, 3.61]
5-2	15.26	[13.78, 16.74]	0.01	15.75	[14.58, 16.92]	0.04	14.36	[4.69, 6.83]	4.78	5.90	[4.68, 7.12]	8.82	[7.59, 10.05]
5-3	16.03	[14.58, 17.48]	0.02	18.84	[17.52, 20.16]	0.06	15.53	[5.25, 8.61]	6.78	3.12	[1.97, 4.27]	17.56	[16.27, 18.85]
6-1	15.95	[14.25, 17.65]	0.02	15.39	[13.65, 17.13]	0.08	14.58	[3.64, 6.12]	8.84	8.59	[7.05, 10.13]	5.26	[3.88, 6.64]
6-2	16.98	[15.74, 18.22]	0.03	18.28	[16.67, 19.89]	0.11	15.64	[4.51, 7.37]	10.84	7.89	[6.70, 9.08]	14.44	[13.13, 15.75]
6-3	17.06	[15.30, 18.82]	0.03	21.61	[20.51, 22.71]	0.17	15.83	[5.00, 7.26]	18.84	7.21	[5.52, 8.90]	26.74	[25.53, 27.95]
7-1	20.68	[19.03, 22.33]	0.05	19.97	[18.39, 21.55]	0.21	17.61	[4.52, 7.70]	20.97	14.85	[13.84, 15.86]	11.81	[10.4, 13.22]
7-2	21.24	[19.94, 22.54]	0.05	22.61	[21.24, 23.98]	0.22	18.84	[5.86, 8.82]	21.25	11.30	[9.81, 12.79]	16.67	[15.64, 17.70]
7-3	22.88	[21.14, 24.62]	0.05	25.57	[23.98, 27.16]	0.27	20.26	[7.44, 10.08]	27.98	11.45	[10.21, 12.69]	20.77	[19.52, 22.02]
8-1	24.33	[23.02, 25.64]	0.07	23.84	[22.41, 25.27]	0.22	22.36	[6.84, 10.28]	22.16	8.10	[6.37, 9.83]	6.21	[5.12, 7.30]
8-2	25.25	[23.66, 26.84]	0.07	26.55	[25.30, 27.80]	0.26	23.88	[9.06, 11.10]	26.15	5.43	[4.23, 6.63]	10.06	[8.63, 11.49]
8-3	26.77	[25.46, 28.08]	0.07	29.37	[27.92, 30.82]	0.31	25.37	[10.22, 12.92]	31.16	5.23	[3.86, 6.60]	13.62	[12.33, 14.91]
Mean										8.04		12.86	

TH_R, TH_C is the mean throughput time of the random and closest assignment rule over 100 demand sets, respectively. ϵ_T^R and ϵ_T^C is the mean improvement percentage of throughput time of the Tabu search-based assignment rule over that of the random and closest assignment rule, respectively.

- For a low demand rate (such as the instances with '1-1' or '2-2'), the closest assignment rule provides a smaller throughput time than the random assignment rule, but the situation reverses for a high demand rate (such as the instance with '3-3'). This result can be explained as that while the closest assignment rule can reduce the robot travel time by minimizing the distance

Table 10

Comparison between the Tabu search-based assignment rule and the random and closest assignment rules in large instances.

Inst.	Random (s)			Closest (s)			TS (s)			Improvement percentage (%)			
	TH_R	CI_{TH_R}	CPU	TH_C	CI_{TH_C}	CPU	TH_T	CI_{TH_T}	CPU	ϵ_T^R	$CI_{\epsilon_T^R}$	ϵ_T^C	$CI_{\epsilon_T^C}$
9-1	31.38	[30.30, 32.46]	0.08	28.53	[27.49, 29.57]	0.26	28.39	[12.84, 14.94]	26.37	9.53	[7.90, 11.16]	0.49	[0.77, 1.75]
9-2	31.99	[30.34, 33.64]	0.08	30.16	[28.62, 31.70]	0.27	29.04	[13.56, 15.52]	28.37	9.22	[7.91, 10.53]	3.71	[2.49, 4.93]
9-3	32.61	[31.33, 33.89]	0.09	32.5	[31.31, 33.69]	0.29	30.19	[14.32, 17.06]	29.38	7.42	[6.38, 8.46]	7.11	[5.68, 8.54]
10-1	32.66	[30.95, 34.37]	0.11	31.07	[29.47, 32.67]	0.33	30.18	[12.86, 15.7]	33.55	7.59	[6.26, 8.92]	2.86	[1.37, 4.35]
10-2	33.32	[32.25, 34.39]	0.12	33.46	[31.99, 34.93]	0.35	32.29	[14.93, 17.85]	35.21	3.09	[1.36, 4.82]	3.50	[1.77, 5.23]
10-3	34.77	[33.01, 36.53]	0.11	35.91	[34.59, 37.23]	0.39	33.78	[16.31, 19.45]	38.56	2.85	[1.28, 4.42]	5.93	[4.65, 7.21]
11-1	36.33	[35.00, 37.66]	0.16	33.51	[32.14, 34.88]	0.31	32.94	[13.85, 17.03]	42.77	9.33	[7.91, 10.75]	1.70	[0.14, 3.26]
11-2	36.84	[35.20, 38.48]	0.14	36.19	[35.19, 37.19]	0.36	34.18	[15.18, 18.18]	42.79	7.22	[6.22, 8.22]	5.55	[3.80, 7.30]
11-3	37.44	[35.68, 39.20]	0.15	39.24	[37.71, 40.77]	0.42	35.13	[16.00, 19.26]	45.78	6.17	[4.91, 7.43]	10.47	[9.03, 11.91]
12-1	37.38	[35.78, 38.98]	0.15	35.77	[34.29, 37.25]	0.35	35.57	[15.10, 18.04]	47.91	4.84	[3.59, 6.09]	0.56	[0.64, 1.76]
12-2	38.04	[36.84, 39.24]	0.16	37.49	[35.82, 39.16]	0.39	36.46	[16.06, 18.86]	52.92	4.15	[3.03, 5.27]	2.75	[1.06, 4.44]
12-3	38.75	[37.73, 39.77]	0.16	42.77	[41.41, 44.13]	0.47	37.88	[17.82, 19.94]	56.92	2.25	[1.17, 3.33]	11.43	[10.44, 12.42]
Mean										6.14		4.67	

Table 11

Comparison between the Tabu search-based assignment rule and the Gurobi solver-based assignment rule in medium and large instances.

Inst.	Gurobi				TS			Improvement	
	TH_G (s)	CI_{TH_G} (s)	Time limit(s)	Optimal ratio	TH_T (s)	CI_{TH_T} (s)	CPU(s)	ϵ_T^G (%)	$CI_{\epsilon_T^G}$ (%)
5-1	13.12	[12.50, 13.74]	3600	93	13.14	[3.21, 5.87]	3.77	0.15	[0.11, 0.20]
5-2	14.35	[13.97, 14.73]	3600	97	14.36	[4.69, 6.83]	4.78	0.07	[0.02, 0.12]
5-3	15.51	[14.98, 16.04]	3600	94	15.53	[5.25, 8.61]	6.78	0.13	[0.08, 0.18]
6-1	14.55	[14.11, 14.99]	3600	92	14.58	[3.64, 6.12]	8.84	0.21	[0.20, 0.22]
6-2	15.63	[14.89, 16.37]	3600	91	15.64	[4.51, 7.37]	10.84	0.06	[0.02, 0.10]
6-3	15.81	[15.54, 16.08]	3600	87	15.83	[5.00, 7.26]	18.84	0.13	[0.12, 0.14]
7-1	17.59	[16.99, 18.19]	3600	79	17.61	[4.52, 7.70]	20.97	0.11	[0.10, 0.12]
7-2	18.83	[18.23, 19.43]	3600	68	18.84	[5.86, 8.82]	21.25	0.05	[0.02, 0.08]
7-3	20.24	[19.66, 20.82]	3600	72	20.26	[7.44, 10.08]	27.98	0.10	[0.09, 0.11]
8-1	22.38	[21.84, 22.92]	3600	56	22.36	[6.84, 10.28]	22.16	-0.09	[-0.12, -0.06]
8-2	23.91	[23.20, 24.62]	3600	53	23.88	[9.06, 11.10]	26.15	-0.13	[-0.15, -0.10]
8-3	25.41	[25.09, 25.73]	3600	45	25.37	[10.22, 12.92]	31.16	-0.16	[-0.19, -0.12]
9-1	28.45	[27.44, 29.46]	3600	51	28.39	[12.84, 14.94]	26.37	-0.21	[-0.23, -0.19]
9-2	29.27	[28.32, 30.22]	3600	53	29.04	[13.56, 15.52]	28.37	-0.79	[-0.81, -0.76]
9-3	30.55	[29.44, 31.66]	3600	46	30.19	[14.32, 17.06]	29.38	-1.18	[-1.21, -1.15]
10-1	31.01	[29.99, 32.03]	3600	37	30.18	[12.86, 15.7]	33.55	-2.68	[-2.71, -2.64]
10-2	32.85	[32.13, 33.57]	3600	36	32.29	[14.93, 17.85]	35.21	-1.70	[-1.73, -1.68]
10-3	33.98	[33.25, 34.71]	3600	34	33.78	[16.31, 19.45]	38.56	-0.59	[-0.62, -0.56]
11-1	33.17	[32.01, 34.33]	3600	12	32.94	[13.85, 17.03]	42.77	-0.69	[-0.74, -0.65]
11-2	35.05	[34.21, 35.89]	3600	14	34.18	[15.18, 18.18]	42.79	-2.48	[-2.52, -2.44]
11-3	36.28	[35.76, 36.80]	3600	12	35.13	[16.00, 19.26]	45.78	-3.17	[-3.21, -3.13]
12-1	36.47	[35.54, 37.40]	3600	6	35.57	[15.10, 18.04]	47.91	-2.47	[-2.51, -2.42]
12-2	37.08	[36.57, 37.59]	3600	7	36.46	[16.06, 18.86]	52.92	-1.67	[-1.71, -1.63]
12-3	38.44	[37.53, 39.35]	3600	3	37.88	[17.82, 19.94]	56.92	-1.46	[-1.51, -1.41]

Note: TH_G is the mean throughput time of Gurobi solver over the 100 demand sets. Optimal ratio is the percentage of the optimal solutions obtained by the Gurobi over 100 demand sets. $\epsilon_T^G = \frac{TH_G - TH_T}{TH_T} \cdot 100\%$ is the improvement percentage of the Gurobi-based assignment rule over the Tabu search-based assignment rule.

between the drop-off points and the loading stations, it may assign too many destinations to a loading station that produces a long waiting time there.

- The Tabu search algorithm significantly outperforms both the random and the closest assignment rules, in terms of system throughput time. The improvement percentage of the Tabu search algorithm over the closest assignment rule significantly increases with the increase of the demand rate of sorting orders. This also reflects that the closest assignment may worsen the waiting process of orders at loading stations, especially for high demand instances. Our Tabu search algorithm aims at finding a balance between the waiting time at the loading stations and the travel time between the loading stations and the drop-off points.
- While the Gurobi solver slightly outperforms the Tabu search algorithm in some medium instances, it is basically dominated by the Tabu search algorithm in large instances. Moreover, the percentage of optimal solutions obtained by the Gurobi solver over the 100 demands sets is significantly decreasing with the increase of the system size.
- The cost comparison results show that the Tabu search algorithm can reduce the number of loading stations required, compared with both the random and the closest assignment rules (Table 12). The manual cost saving is significant in large systems, especially the large instances.

Table 12

Manual cost comparison between the Tabu search-based assignment rule and the random and closest assignment rules.

Medium instances						Large instances					
Inst.	n_C	n_R	n_T	ϵ_T^C (%)	ϵ_T^R (%)	Inst.	n_C	n_R	n_T	ϵ_T^C (%)	ϵ_T^R (%)
5-1	3	3	3	0.00	0.00	9-1	10	12	8	20.00	33.33
5-2	3	3	3	0.00	0.00	9-2	11	12	8	27.27	33.33
5-3	3	3	3	0.00	0.00	9-3	11	12	9	18.18	25.00
6-1	4	5	4	0.00	20.00	10-1	13	15	11	15.38	26.67
6-2	5	5	4	20.00	20.00	10-2	13	15	12	7.69	20.00
6-3	5	5	4	20.00	20.00	10-3	14	15	12	14.29	20.00
7-1	5	6	4	20.00	33.33	11-1	14	16	12	14.29	25.00
7-2	5	6	4	20.00	33.33	11-2	14	16	12	14.29	25.00
7-3	6	6	5	16.67	16.67	11-3	15	16	13	13.33	18.75
8-1	8	9	7	12.50	22.22	12-1	15	18	13	13.33	27.78
8-2	8	9	7	12.50	22.22	12-2	16	18	14	12.50	22.22
8-3	9	9	7	22.22	22.22	12-3	16	18	14	12.50	22.22

Note: n_C, n_R, n_T refers to the number of loading stations used by the closest, random, Tabu search-based assignment rule, respectively. $\epsilon_T^C, \epsilon_T^R$ is the cost saving percentage of the Tabu search-based assignment rule over the closest and random assignment rule, respectively. For instance, $\epsilon_T^C = \frac{n_C - n_T}{n_C} \cdot 100\%$.

Table 13

Validation results of Tabu search algorithm by a real case.

Date (M/D/Y)	Throughput time			Waiting time at loading station			Loading station number		
	TH_D	TH_T	ϵ_{TH}^D (%)	$\bar{W}_{Is}^D$	$\bar{W}_{Is}^T$	ϵ_W^D (%)	n_D	n_T	ϵ_n^D (%)
11/5/18	26.25	21.66	17.50	5.61	4.82	14.03	6	5	16.67
11/11/18	30.82	25.32	17.84	11.55	5.25	54.56	6	6	0.00
11/15/18	41.17	28.97	29.64	12.27	4.98	59.40	6	6	0.00
11/20/18	40.22	30.97	22.99	7.51	4.00	46.70	6	5	16.67
11/22/18	40.56	31.41	22.56	11.35	5.07	55.27	6	6	0.00
11/28/18	45.84	30.96	32.46	7.39	4.44	39.95	6	6	0.00
12/7/18	40.66	30.00	26.22	9.59	5.00	47.86	5	4	20.00
12/13/18	37.91	29.30	22.70	7.44	4.68	37.15	6	5	16.67
12/19/18	50.48	32.48	35.65	12.81	5.03	60.77	6	5	16.67
11/25/18	48.41	32.33	33.21	9.75	4.57	53.06	6	5	16.67
12/31/18	38.18	29.29	23.28	8.50	5.16	39.32	6	6	0.00
1/1/19	41.96	28.18	32.85	11.89	4.50	62.17	6	5	16.67
1/7/19	29.34	20.43	30.37	7.01	3.88	44.65	5	4	20.00
1/19/19	24.34	22.79	6.38	5.79	4.44	23.28	6	5	16.67
1/22/19	33.33	26.68	19.94	7.88	4.80	39.08	5	4	20.00
1/25/19	47.98	29.70	38.11	9.52	5.59	41.22	6	5	16.67
1/28/19	35.04	28.86	17.64	7.01	4.37	37.71	6	5	16.67
2/12/19	41.50	26.05	37.24	10.14	4.17	58.92	5	4	20.00
2/13/19	27.31	20.05	26.57	4.75	4.50	5.23	5	2	60.00
2/14/19	31.42	21.78	30.67	7.98	3.82	52.05	5	4	20.00
2/15/19	30.43	25.20	17.19	8.46	4.86	42.54	4	4	0.00
2/20/19	37.99	25.41	33.10	9.17	4.04	55.92	6	5	16.67
2/25/19	30.66	26.18	14.62	10.74	6.50	39.44	7	5	28.57
3/1/19	30.73	23.87	22.33	7.75	3.99	48.58	6	5	16.67
3/7/19	38.96	27.47	29.48	10.07	6.00	40.43	6	5	16.67
3/13/19	30.53	24.61	19.40	8.68	4.75	45.24	6	6	0.00
3/20/19	40.44	30.36	24.92	15.26	6.41	58.03	7	5	28.57
3/25/19	32.24	27.21	15.62	7.65	4.61	39.76	7	6	14.29
3/27/19	35.84	23.11	35.52	13.34	5.10	61.77	6	5	16.67
3/30/19	24.13	23.05	4.47	6.45	5.29	17.95	6	4	33.33
Mean value			24.68			44.07			16.05

6.2. Evaluating TS by a real case

In this section, we use the data provided by Deppon Express in Shanghai China to verify the Tabu search algorithm in both terms of throughput time and manual costs. We randomly selected 30 days in 5 consecutive months from November 2018 to March 2019 for verification. The comparison results are included in Table 13. It appears that the Tabu search algorithm can improve the throughput time by an average of 35%, the expected waiting time at loading stations by an average of 55%, and reduce the manual cost by an average of 16%.

Furthermore, from the perspective of manual cost, when manual cost is not improved, that is, when δ_C^D equals to 0, the waiting time improvement percentage of Tabu search algorithm is higher than average level. This can be explained as that when the drop-off points is too concentrately assigned to a certain loading station, it leads to an increase in the utilization rate of this loading station, which also means that manual resources will be more fully utilized, so our optimization algorithm not be too obvious in manual

Table 14Throughput time comparison between the Tabu search-based assignment rule and the random and closest assignment rule in large instances with $cv = 0.25$.

Inst.	Random (s)			Closest (s)			TS (s)			Improvement percentage (%)			
	TH_R	CI_{TH_R}	CPU	TH_C	CI_{TH_C}	CPU	TH_T	CI_{TH_T}	CPU	ϵ_T^R	$CI_{\epsilon_T^R}$	ϵ_T^C	$CI_{\epsilon_T^C}$
1-1	31.62	[31.33, 33.09]	0.13	29.40	[29.08, 31.03]	0.13	28.75	[28.4, 30.53]	27.33	9.08	[8.8, 10.52]	2.21	[1.95, 3.51]
1-2	32.19	[31.98, 33.24]	0.14	31.08	[30.83, 32.37]	0.14	29.46	[29.15, 31.03]	28.70	8.48	[8.24, 9.70]	5.21	[4.98, 6.39]
1-3	32.75	[32.38, 34.66]	0.13	33.29	[33.07, 34.39]	0.13	30.64	[30.40, 31.88]	29.67	6.44	[6.08, 8.30]	7.96	[7.72, 9.17]
2-1	33.02	[32.79, 34.18]	0.17	32.07	[31.81, 33.40]	0.17	30.45	[30.13, 32.09]	34.39	7.78	[7.55, 8.94]	5.05	[4.77, 6.45]
2-2	33.80	[33.45, 35.57]	0.16	34.09	[33.82, 35.46]	0.16	32.61	[32.32, 34.09]	35.94	3.52	[3.25, 4.90]	4.34	[4.01, 6.01]
2-3	35.01	[34.79, 36.16]	0.18	36.52	[36.15, 38.42]	0.18	34.16	[33.80, 36.00]	38.76	2.43	[2.11, 4.06]	6.46	[6.22, 7.68]
3-1	36.81	[36.43, 38.77]	0.17	34.39	[34.07, 36.03]	0.17	33.05	[32.81, 34.28]	43.66	10.21	[9.97, 11.43]	3.90	[3.62, 5.33]
3-2	37.12	[36.89, 38.27]	0.18	36.89	[36.68, 37.98]	0.18	34.51	[34.30, 35.58]	43.00	7.03	[6.78, 8.31]	6.45	[6.25, 7.49]
3-3	37.73	[37.46, 39.12]	0.20	40.11	[39.81, 41.67]	0.2	35.59	[35.21, 37.53]	46.73	5.67	[5.32, 7.47]	11.27	[10.93, 13.02]
4-1	37.78	[37.47, 39.37]	0.22	36.76	[36.42, 38.51]	0.22	35.92	[35.54, 37.85]	48.90	4.92	[4.70, 6.03]	2.29	[1.97, 3.91]
4-2	38.43	[38.18, 39.72]	0.21	38.49	[38.28, 39.57]	0.21	36.68	[36.42, 38.03]	53.63	4.55	[4.35, 5.59]	4.70	[4.40, 6.23]
4-3	39.13	[38.80, 40.82]	0.23	43.40	[43.06, 45.13]	0.23	38.28	[38.01, 39.65]	57.77	2.17	[1.83, 3.89]	11.80	[11.43, 13.7]

Table 15Throughput time comparison between the Tabu search-based assignment rule and the random and closest assignment rule in large instances with $cv = 0.5$.

Inst.	Random (s)			Closest (s)			TS (s)			Improvement percentage (%)			
	TH_R	CI_{TH_R}	CPU	TH_C	CI_{TH_C}	CPU	TH_T	CI_{TH_T}	CPU	ϵ_T^R	$CI_{\epsilon_T^R}$	ϵ_T^C	$CI_{\epsilon_T^C}$
1-1	32.47	[32.16, 34.07]	0.20	30.20	[29.08, 31.03]	0.20	29.27	[29.96, 31.44]	28.10	9.86	[9.64, 11.01]	3.08	[2.71, 4.99]
1-2	33.06	[32.80, 34.41]	0.22	31.97	[30.83, 32.37]	0.21	30.34	[31.77, 33.01]	29.37	8.23	[7.93, 9.75]	5.10	[4.77, 6.79]
1-3	33.44	[33.24, 34.46]	0.23	34.25	[33.07, 34.39]	0.22	31.47	[34.00, 35.53]	30.48	5.89	[5.57, 7.53]	8.12	[7.85, 9.52]
2-1	33.87	[33.51, 35.69]	0.25	32.91	[31.81, 33.40]	0.24	31.18	[32.71, 33.95]	35.04	7.94	[7.56, 9.89]	5.26	[5.03, 6.43]
2-2	34.32	[34.04, 35.77]	0.26	34.84	[33.82, 35.46]	0.21	33.12	[34.58, 36.18]	36.65	3.50	[3.29, 4.57]	4.94	[4.57, 6.85]
2-3	35.56	[35.32, 36.78]	0.26	37.05	[36.15, 38.42]	0.28	35.12	[36.74, 38.62]	39.60	1.24	[0.88, 3.10]	5.21	[4.89, 6.87]
3-1	37.77	[37.48, 39.27]	0.23	34.95	[34.07, 36.03]	0.26	33.99	[34.57, 36.91]	44.41	10.01	[9.63, 11.97]	2.75	[2.38, 4.64]
3-2	38.01	[37.62, 39.99]	0.23	37.62	[36.68, 37.98]	0.24	35.22	[37.32, 39.13]	43.80	7.34	[7.02, 8.96]	6.38	[6.16, 7.50]
3-3	38.38	[38.04, 40.12]	0.26	40.93	[39.81, 41.67]	0.27	36.16	[40.69, 42.16]	47.44	5.78	[5.47, 7.36]	11.65	[11.45, 12.67]
4-1	38.65	[38.32, 40.31]	0.30	37.26	[36.42, 38.51]	0.29	36.54	[36.98, 38.70]	49.57	5.46	[5.15, 7.02]	1.93	[1.62, 3.53]
4-2	39.30	[39.09, 40.38]	0.28	39.20	[38.28, 39.57]	0.30	37.23	[38.86, 40.92]	54.25	5.27	[4.97, 6.80]	5.03	[4.77, 6.38]
4-3	39.67	[39.32, 41.48]	0.31	44.33	[43.06, 45.13]	0.30	39.15	[43.96, 46.21]	58.29	1.31	[0.98, 2.98]	11.69	[11.36, 13.38]

cost. On the other hand, some loading stations will be assigned to drop-off points with larger arrival rates, and some loading stations are assigned to the drop-off points with smaller arrival rates, which will lead to the unbalanced utilization of loading stations. Due to the large workload of some loading stations, the waiting time of these loading stations would increase sharply. The Tabu search algorithm takes into account the balance of the workload of the loading stations, and there will be no excessive congestion of the loading stations. Therefore, the use of the optimization algorithm has a significant improvement of the waiting time.

6.3. Sensitivity analysis on the distribution of inter-arrival times of sorting orders

In this section, we consider two general distributions of inter-arrival times of sorting orders in the large instances to evaluate the effect of the distribution of order arrival process on the system performance. We assume that the arrival process of sorting orders of a specific destination follows a general distribution with a mean value λ_{d_i} and a coefficient of variation cv . Without loss of generality, we take λ_{d_i} from Table 7 and cv by two values 0.25 and 0.5. The throughput time comparison result of the large instance under the case $cv = 0.25$ and the case $cv = 0.5$ is presented in Table 14 and Table 15, respectively. It shows that the variation of the inter-arrival times of sorting orders (cv) has a slight effect on the throughput time of the RSS that follows the random, closest or the Tabu search-based assignment rule.

7. Conclusion and future work

This paper examines the assignment problem of parcels to loading stations in a robotic sorting system that uses autonomous robots to transport parcels sorted to the loading stations located at the perimeter of the system to the drop-off points located in the middle of the system. While assigning more parcels to the loading station close to the drop-off points can reduce the robot travel distance, it may also lead to serious congestion at the loading station and on the conveyor that sorts parcels to loading stations. We aim to balance the trade-off by optimizing the assignment of parcels to loading stations in such a system.

We first build an open queueing network to estimate the system performance of the system, including the system throughput time, the expected waiting time at loading stations and the utilization of loading stations. This model can capture the congestion on the conveyor that sorts parcels to loading stations. We design a decomposition method to solve the model and validate the accuracy of the performance estimation model by simulation. Then, we build an integer programming model for the assignment problem, that takes the minimization of system throughput time as objective and the maximum service capacity of loading stations as the key constraints. We prove that this model is NP-hard by transforming it to the order batching problem, and design a heuristic that

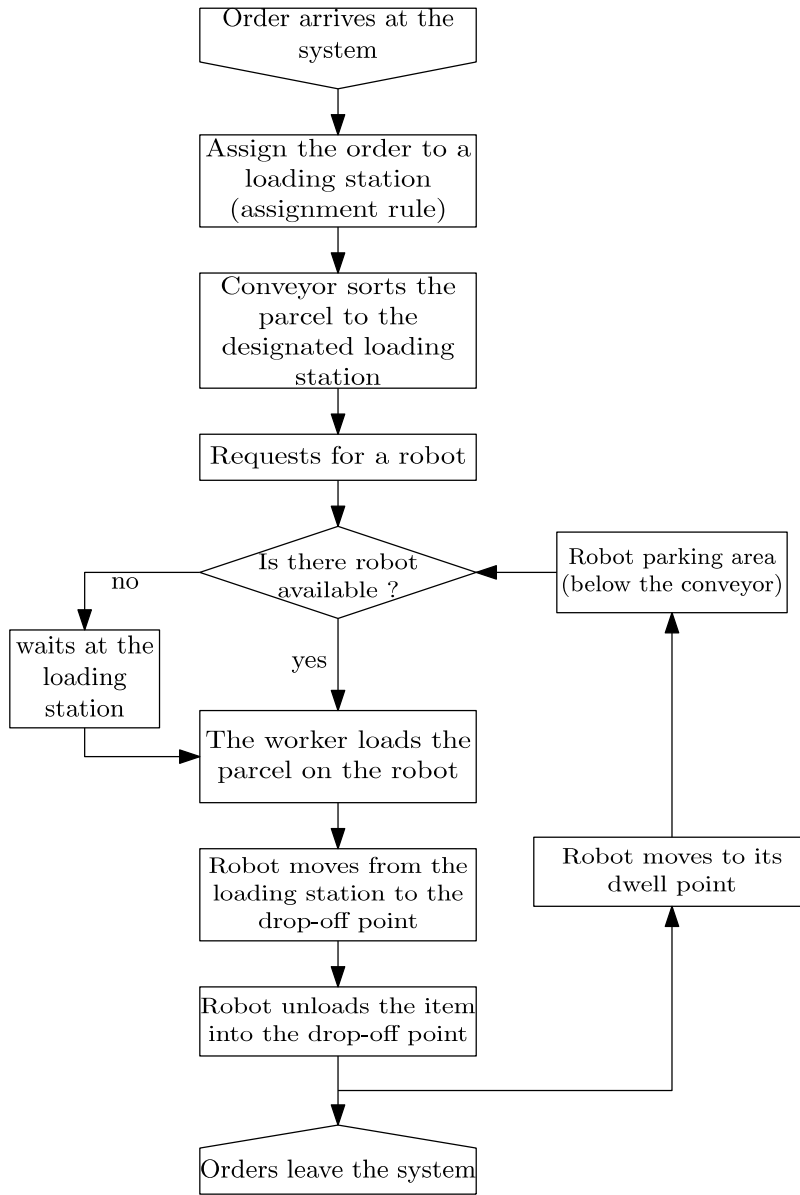


Fig. 8. The flowchart of the simulation model of RSSs.

combines a BFD procedure and a Tabu search algorithm to solve the model. We have compared the performance of the Tabu search algorithm with that of the random assignment rule, the closest assignment rule and the Gurobi-based assignment rule in medium and large instances. The results show that our approach significantly outperforms both the random and the closest assignment rules in both terms of system throughput time and the manual cost, and produces a better throughput time than the Gurobi solver in large instances. Moreover, we have implemented the heuristic in a real case. The results show that our approach can lower the system throughput time by about 24% and reduce the manual cost by about 16%. We at last conduct a sensitivity analysis on the arrival distribution of sorting orders and find that the variation of the inter-arrival times of sorting orders slightly affect the system throughput time.

For future study, we are interested at the allocation of demands among drop-off points, that may affect the robot travel distance between the loading stations and the drop-off points and also the possible congestion on the route of the robot.

Robotic Sorting system

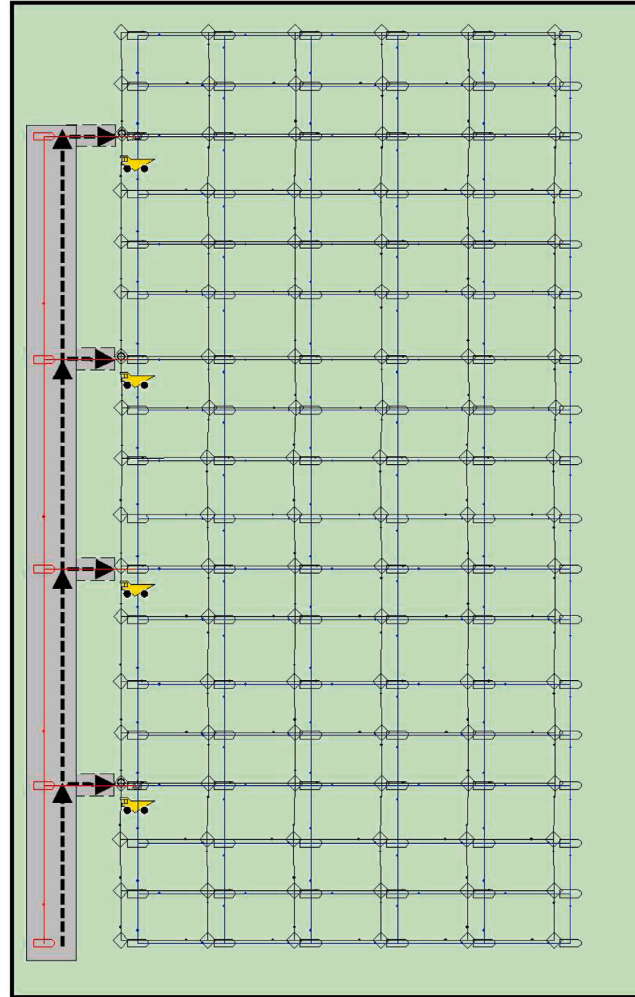


Fig. 9. The animation view of the simulation model of RSSs.

Acknowledgment

This study was supported by the National Natural Science Foundation of China [grant number 72171233, 71801225, 71620107002, and 71821001], the National Social Science Foundation of China [grant number 20BJY188, 21BGL035]. We would like to express our thanks to Deppon Express for providing the AGV sorting system data.

Appendix A. Simulation model

We present the flowchart and the animation view (top view) of the simulation model of RSSs in Fig. 8 and Fig. 9, respectively.

Appendix B. A semi-open queueing network for the RSS

Fig. 10 depicts the semi-open queueing network for the RSS with R robots. The order is first sorted by the conveyor to the loading station through the nodes $\mu_{c_i}, i = 1, 2, \dots, n$. The order then requires a robot at the synchronization node that consists of a order queue where the arrived order waits for a robot and a robot queue where an idle robot waits for an order. The remaining sorting processes are the same as the OQN (Fig. 3). After that, the robot returns to the dwell area at the node $\mu_{d,ls}$, noting that the idle robot will park nearby the loading stations.

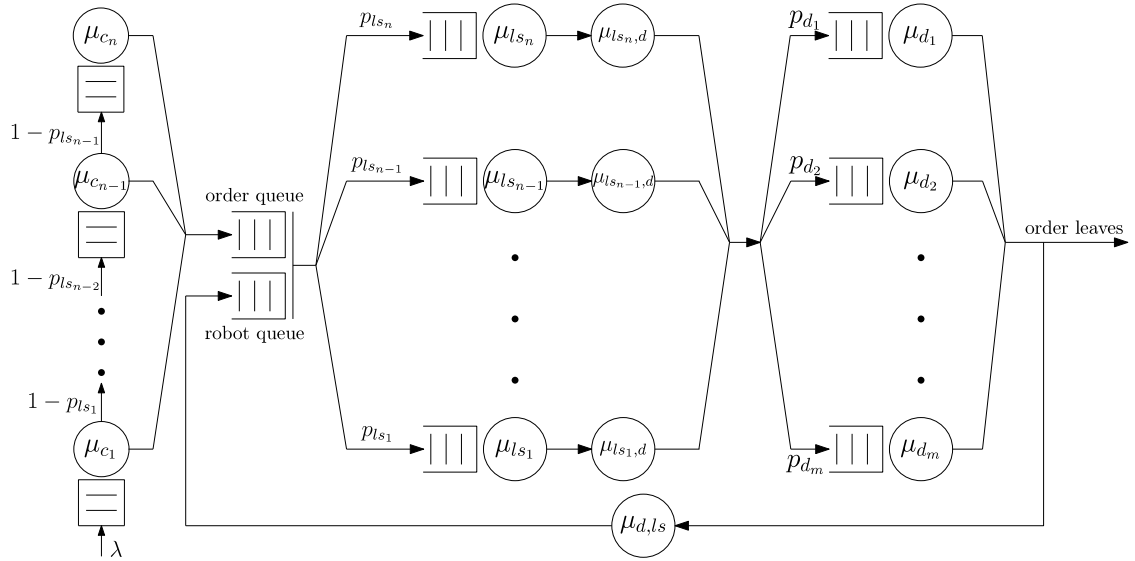


Fig. 10. A semi-open queueing network for the RSS.

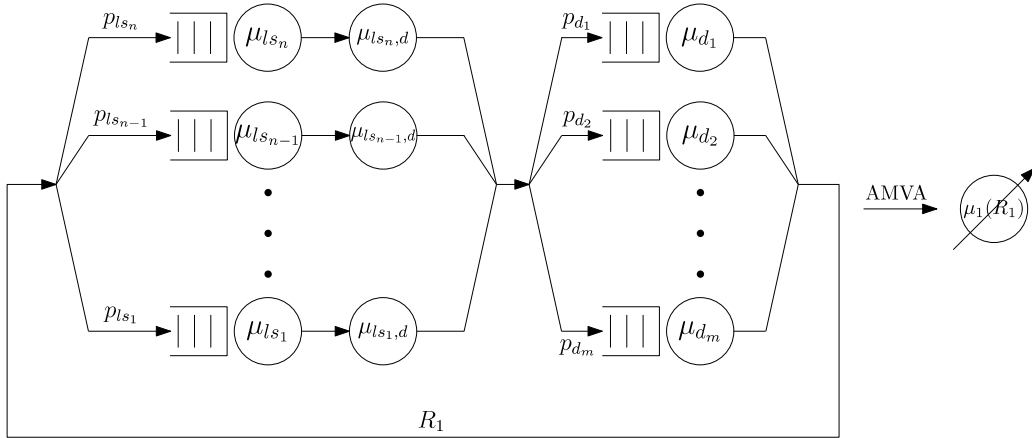


Fig. 11. A closed queueing network for the sorting part of the RSS.

We use a decomposition method to analyze the performance measures of the semi-open queueing network, that consists of the following steps:

1. We first short circuit the sorting processes (including nodes $\mu_{ls_k}, \mu_{ls_{k,d}}, \mu_{d_j}, i = 1, 2, \dots, n, j = 1, 2, \dots, m$) as a closed queueing network (Fig. 11), and then replace it by a load-dependent serve node $\mu_1(R_1) = TC_1(R_1)$ where $TC_1(R_1)$ is the throughput capacity of the CQN derived by the approximate mean value analysis (AMVA) method.
2. We then transfer the original SOQN into the reduced SOQN (Fig. 12) and analyze its performance by the matrix-geometric method.

Appendix C. An iterative procedure for performance estimation of the OQN for the sorter system

We first define the notations used in the iterative procedure in Table C.16. We initialize the time-instant starving probability $p_s^*(j, j+1)$ and blocking probability $p_b^*(j, j+1)$ for each subsystem, then conduct the following iterative procedures:

1. Adjusting the virtual arrival process: the virtual arrival rate of the subsystem $(j, j+1)$ can be represented by the effective service time of station j when two successive items are served. The expected virtual arrival rate and the squared coefficient

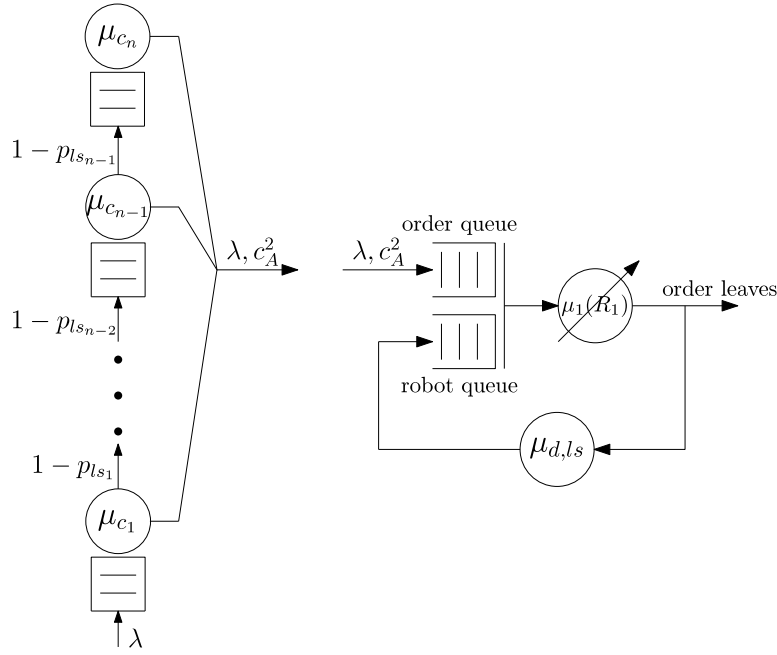


Fig. 12. The reduced semi-open queueing network of the RSS.

Table C.16

Notations used in the iterative procedure.

Notation	Description
$(j, j+1)$	a two-station subsystem that consists of a preceding processing station j , a downstream processing station $j+1$ and their intermediate buffer $B_{j,j+1}$
$\mu_u(j, j+1), \zeta_u(j, j+1)$	the virtual arrival rate and squared coefficient of variation of inter arrival times between two customers at the subsystem $(j, j+1)$
$\mu_d(j, j+1), \zeta_d(j, j+1)$	the virtual departure rate and squared coefficient of variation of inter departure times between two customers at the subsystem $(j, j+1)$
$p_e(j, j+1)$	the state probability of the subsystem $(j, j+1)$
$p_b^*(j, j+1)$	the probability that, at the instant of a service completion at station j , it is blocked by station $j+1$
$p_s^*(j, j+1)$	the probability that, at the instant of a service completion at station $j+1$, this station is starving for parts from station j
$p_p(j, j+1)$	time-average probability that the subsystem $(j, j+1)$ is filled up with $B_{j,j+1} + 2$ customers
$p_e(j, j+1)$	time-average probability that the subsystem $(j, j+1)$ is empty
X	throughput of the whole system

of variation of inter-arrival time of the subsystem $(j, j+1)$ can be obtained by the following equations:

$$\frac{1}{\mu_u(j, j+1)} = \frac{1}{\mu_j} + (1 - p_{ls_j}) \cdot p_s^*(j-1, j) \cdot \frac{1}{\mu_u(j-1, j)}, j = 2, \dots, n-1. \quad (14)$$

$$\zeta_u^2(j, j+1) = (1 - p_{ls_j}) \cdot \mu_u^2(j, j+1) \cdot \left[\frac{c_j^2 + 1}{\mu_j^2} + p_s^*(j-1, j) \cdot \left(\frac{\zeta_u^2(j-1, j) + 1}{\mu_u^2(j-1, j)} + \frac{2}{\mu_j \cdot \mu_u(j-1, j)} \right) \right] - 1, j = 2, \dots, n-1 \quad (15)$$

2. Adjusting the virtual departure process: the virtual departure rate $\mu_d(j, j+1)$ of subsystem $(j, j+1)$ can be represented by the total holding time at station $j+1$, which includes the service time of station $j+1$ and the blocking delay caused by the successor station of station $j+1$. The expected virtual departure rate and the squared coefficient of variation of inter-departure time of the subsystem $(j, j+1)$ can be obtained by the following equations:

$$\frac{1}{\mu_d(j-1, j)} = \frac{1}{\mu_j} + (1 - p_{ls_j}) \cdot p_b^*(j, j+1) \cdot \frac{1}{\mu_d(j, j+1)} \quad (16)$$

$$\zeta_d^2(j-1, j) = (1 - p_{ls_j}) \cdot \mu_d^2(j-1, j) \cdot \left[\frac{c_j^2 + 1}{\mu_j^2} + p_b^*(j, j+1) \cdot \left(\frac{\zeta_d^2(j, j+1) + 1}{\mu_d^2(j, j+1)} + \frac{2}{\mu_j \cdot \mu_d(j, j+1)} \right) \right] - 1, j = 2, \dots, n-1. \quad (17)$$

3. Updating the starving probability (instance and time-average) and blocking probability (instance and time-average) of each subsystem: We take the subsystem $(j, j+1)$ as a $G/G/1/B_{j,j+1}$ queueing system with an arrival rate $\mu_u(j, j+1)$, with a service

rate $\mu_d(j, j+1)$, with a coefficient of variation of the inter-arrival time $\zeta_u(j, j+1)$, with a coefficient of variation of the service time $\zeta_d(j, j+1)$, and with a capacity of $B_{j,j+1}$ customers. We use the approximate method of Buzacott and Shanthikumar (1993) to calculate the time-average state probability of the $G/G/1/B_{j,j+1}$ queueing system, denoted as $p_k(j, j+1)$, where k is the number of customers in the system. We can obtain the starving and blocking probability in average-time situation of the subsystem by Eq. (18)

$$p_s(j, j+1) = p_0(j, j+1), p_b(j, j+1) = p_{B_{j,j+1}+2}(j, j+1). \quad (18)$$

Then, we obtain the starving and blocking probability in time-instant situation of the subsystem $(j, j+1)$ by Eq. (19)

$$p_s^*(j, j+1) = \frac{\mu_u(j, j+1)}{\mu_d(j, j+1)} \cdot \frac{p_s(j, j+1)}{1 - p_s(j, j+1)}, p_b^*(j, j+1) = \frac{\mu_d(j, j+1)}{\mu_u(j, j+1)} \cdot \frac{p_d(j, j+1)}{1 - p_d(j, j+1)}. \quad (19)$$

4. Calculate the throughput of the whole system by Eq. (20), and go back to step one until the change in X is within a pre-specified tolerance ϵ .

$$X = \mu_d(n-1, n) \cdot (1 - p_s(n-1, n)) \quad (20)$$

Algorithm 4 Iterative algorithm for performance measures of conveyor

```

1: repeat
2:   for all nodes  $j = 1, \dots, n-1$  do
3:     Data input:  $\mu_j, c_j, B_{j,j+1}, \epsilon$ 
4:     Initialize the virtual parameters with the values of the original conveyor nodes.
5:     Initialize the starving and blocking probabilities with 0.
6:     Initialize the throughput of the whole system by  $X = \min_{j=1,2,\dots,n} \mu_j$ 
7:   end for
8:   for all nodes  $j = 1, \dots, n-1$  do
9:     Calculate  $\mu_u(j, j+1)$  and  $\zeta_u^2(j, j+1)$  by Eq. (14) and Eq. (15), respectively.
10:  end for
11:  for any nodes  $j = n-1, \dots, 1$  do
12:    Calculate  $\mu_d(j, j+1)$  and  $\zeta_d^2(j, j+1)$  according to Eq. (16) and Eq. (17), respectively.
13:  end for
14:  Obtain the time-average starving and blocking probabilities in by Eq. (18).
15:  Obtain the time-instant starving and blocking probabilities by Eq. (19).
16:  Calculate the throughput of the whole system by Eq. (20)
17: until The change in throughput  $X$  is within a pre-specified threshold  $\epsilon$ .
18: Output:  $X, \mu_d$  and  $\zeta_d^2$  of all the stations.

```

References

- Acar, K., Yalcin, A., Yankov, D., 2012. Robust door assignment in less-than-truckload terminals. *Comput. Ind. Eng.* 63 (4), 729–738.
- Ames, B., 2019. Amazon unveils Xanthus and Pegasus fulfillment robots. DC Velocity, URL <https://www.dcvelocity.com/articles/20190606-amazon-unveils-xanthus-and-pegasus-fulfillment-robots/#.XP8tsesSJVs.twitter>.
- Ascó, A., Jad, A., Burke, E.K., 2014. An analysis of constructive algorithms for the airport baggage sorting station assignment problem. *J. Sched.* 17 (6), 601–619.
- Azadeh, K., De Koster, R., Roy, D., 2019a. Robotized and automated warehouse systems: Review and recent developments. *Transp. Sci.* 53 (4), 917–945.
- Azadeh, K., Roy, D., De Koster, R., 2019b. Design, modeling, and analysis of vertical robotic storage and retrieval systems. *Transp. Sci.* 53 (5), 1213–1234.
- Bolch, G., Greiner, S., De Meer, H., Trivedi, K.S., 2006. Queueing Networks and Markov Chains: Modeling and Performance Evaluation with Computer Science Applications. John Wiley & Sons.
- Boysen, N., Briskorn, D., Fedtke, S., 2019. Automated sortation conveyors: A survey from an operational research perspective. *European J. Oper. Res.* 276 (3), 796–815.
- Boysen, N., Briskorn, D., Tschöke, M., 2013. Truck scheduling in cross-docking terminals with fixed outbound departures. *OR Spectrum* 35 (2), 479–504.
- Boysen, N., Briskorn, D., Emdec, S., 2017. Parts-to-picker based order processing in a rack-moving mobile robots environment. *European J. Oper. Res.* 262 (1), 550–562.
- Boysen, N., Fedtke, S., Weidinger, F., 2018. Optimizing automated sorting in warehouses: The minimum order spread sequencing problem. *European J. Oper. Res.* 270 (1), 386–400.
- Boysen, N., Flidner, M., Scholl, A., 2010. Scheduling inbound and outbound trucks at cross docking terminals. *OR Spectrum* 32 (1), 135–161.
- Briskorn, D., Emde, S., Boysen, N., 2017. Scheduling shipments in closed-loop sortation conveyors. *J. Sched.* 20 (1), 25–42.
- Buzacott, J., Shanthikumar, J., 1993. Stochastic Models of Manufacturing Systems. Prentice Hall.
- Gademann, N., Velde, S., 2005. Order batching to minimize total travel time in a parallel-aisle warehouse. *IIE Trans.* 37 (1), 63–75.
- Jarrah, A.I., Qi, X., Bard, J.F., 2016. The destination-loader-door assignment problem for automated package sorting centers. *Transp. Sci.* 50 (4), 1314–1336.
- Lamballais, T., Roy, D., De Koster, R., 2017. Estimating performance in a robotic mobile fulfillment system. *European J. Oper. Res.* 256 (3), 976–990.
- Lamballais, T., Roy, D., De Koster, M., 2020. Inventory allocation in robotic mobile fulfillment systems. *IIE Trans.* 52 (1), 1–17.
- Li, X., Hua, G., Huang, A., Sheu, J.-B., Cheng, T., Huang, F., 2020. Storage assignment policy with awareness of energy consumption in the Kiva mobile fulfillment system. *Transp. Res. Part E: Logist. Transp. Rev.* 144, 102158.
- Marchet, G., Melacini, M., Perotti, S., Tappia, E., 2012. Analytical model to estimate performances of autonomous vehicle storage and retrieval systems for product totes. *Int. J. Prod. Res.* 50 (24), 7134–7148.
- McWilliams, D.L., 2009. A dynamic load-balancing scheme for the parcel hub-scheduling problem. *Comput. Ind. Eng.* 57 (3), 958–962.

- Meller, R.D., 1997. Optimal order-to-lane assignments in an order accumulation/sortation system. *IIE Trans.* 29 (4), 293–301.
- Merschformann, M., Lamballais, T., De Koster, M., Suhl, L., 2019. Decision rules for robotic mobile fulfillment systems. *Oper. Res. Perspect.* 6, 100128.
- Mountz, M., 2012. Kiva the disrupter. *Harv. Bus. Rev.* 90, 74–80.
- Roy, D., Nigam, S., Adan, I., De Koster, R., Resing, J., 2016. *Mobile Fulfillment Systems: Model and Design Insights*. Erasmus University, the Netherlands.
- Roy, D., Nigam, S., de Koster, R., Adan, I., Resing, J., 2019. Robot-storage zone assignment strategies in mobile fulfillment systems. *Transp. Res. Part E: Logist. Transp. Rev.* 122, 119–142. <http://dx.doi.org/10.1016/j.tre.2018.11.005>, URL <https://www.sciencedirect.com/science/article/pii/S1366554518304307>.
- Tompkins, 2020. T-sort solution revolutionizing unit and parcel sortation. URL <https://www.tompkinsrobotics.com/Solutions/t-Sort>.
- Valle, C.A., Beasley, J.E., 2021. Order allocation, rack allocation and rack sequencing for pickers in a mobile rack environment. *Comput. Oper. Res.* 125, 105090.
- Van Belle, J., Valckenaers, P., Berghe, G.V., Cattrysse, D., 2013. A tabu search approach to the truck scheduling problem with multiple docks and time windows. *Comput. Ind. Eng.* 66 (4), 818–826.
- Wang, Z., Sheu, J.B., Teo, C.P., Xue, G., 2022. Robot scheduling for mobile-rack warehouses: Human–Robot coordinated order picking systems. *Prod. Oper. Manage.* 31 (1), 98–116.
- Weidinger, F., Boysen, N., Briskorn, D., 2018. Storage assignment with rack-moving mobile robots in KIVA warehouses. *Transp. Sci.* 52 (6), 1479–1495.
- Whitt, W., 1983. Performance of the queueing network analyzer. *Bell Syst. Tech. J.*
- Xiang, X., Liu, C., Miao, L., 2018. Storage assignment and order batching problem in kiva mobile fulfillment system. *Eng. Optim.* 50 (11), 1941–1962.
- Zhang, H., 2019. Chinese market boosts technical innovation in logistics. CGTN, URL <https://news.cgtn.com/news/2019-11-10/Chinese-market-boosts-technical-innovation-in-logistics-Lvf4PeRoDm/index.html>.
- Zou, B., Chen, Y., 2020. Assign orders to workstations in robotic sorting systems considering destination information. pp. 343–348, URL <https://www.computer.org/csdl/proceedings-article/icisce/2020/640600a343/1x3ksfT9j1K>.
- Zou, B., De Koster, R., Gong, Y., Xu, X., Shen, G., 2021. Robotic sorting systems: Performance estimation and operating policies analysis. *Transp. Sci.* 55 (6), 1430–1455.
- Zou, B., De Koster, R., Xu, X., 2018. Operating policies in robotic-compact storage and retrieval systems. *Transp. Sci.* 52 (4), 788–811.
- Zou, B., Gong, Y., Xu, X., Yuan, Z., 2017. Assignment rules in robotic mobile fulfillment systems for online retailers. *Int. J. Prod. Res.* 55 (20), 6175–6192.